

THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

Sustainability as a Software Engineering Problem

Aryan Thanki

Supervised by Judy Bowen

October 2025

This report is submitted in partial fulfilment of the requirements for the degree of Bachelor of Engineering with Honours (BE(Hons)) at The University of Waikato.

Abstract

This research investigates sustainability viewed as a software engineering problem, understanding how software abandonment, maintenance practices and professional awareness affect long-term digital sustainability. A mixed methods approach was used, combining large-scale quantitative scraping of three digital platforms, the Google Play Store, Apple App Store and GitHub, with a qualitative survey targeting software developers and engineers in New Zealand. Scraping analysis revealed a widespread level of inactivity across all platforms, where a significant number of applications remained outdated and no longer maintained, showcasing an absence of lifecycle management. Survey findings complement this by showing a limited awareness for sustainable software engineering principles, with many participants likening sustainability solely to energy efficiency rather than full lifecycle responsibility.

An Insights Matrix was created to combine data from both methods and literature, identifying themes of awareness, organisational governance, measurement, and education. This analysis revealed that lack of sustainability is not mainly a technical fault but arises from weak organisational structures, absence of standardised metrics and lack of training. However, strong interest exists with many professionals enthusiastic to learn about sustainable development, which indicates a foundation for improvement already exists.

Overall, this study shows a substantial gap between theory and practice in green software engineering. It emphasises how measurable sustainability indicators, structured education and governance at the policy level are needed for sustainability to transfer from an aspirational idea into a practical attribute that guides software development practice.

Table of Contents

1. Introduction 7
   1.1 Background and Motivation 7
   1.2 Problem Definition 7
   1.3 Scope of Study 8
2. Background and Literature Review 10
   2.1 Sustainability in Computing 10
   2.2 Foundations of Green Software Engineering 10
   2.3 Software Lifecycle and Abandonment 11
   2.4 Adoption Challenges in Industry 12
3. Methodology 14
   3.1 Research Design 14
   3.2 App Store and Repository Scraping 14
      3.2.1 Google Play Store 15
      3.2.2 Apple App Store 15
      3.2.3 GitHub Repositories 16
   3.3 Green Software Adoption Survey 16
      3.3.1 Ethical Process 17
      3.3.2 Survey Design 17
      3.3.3 Data Collection 18
4. Implementation 19
   4.1 Technology Stack 19
   4.2 App Store and Repository Scraping 19
      4.2.1 Google Play Store 20
      4.2.2 Apple App Store 20
      4.2.3 GitHub Repositories 21
   4.3 Green Software Adoption Survey 21
      4.3.1 Survey Execution 21
      4.3.2 Data Handling and Analysis 22
5. Results 23
   5.1 Play Store Results 23
   5.2 App Store Results 23
   5.3 GitHub Repository Results 24
   5.4 Survey Results 24
6. Discussion 27
   6.1 Insights Matrix and Comparative Analysis 27
   6.2 Key Challenges in Sustainable Software Engineering Adoption 31

7. Conclusion 33
7.1 Future Work 33
References 34
Appendix A 37
Literature Review 37
Appendix B 60
Official Ethics Approval Letter 60

List of Figures

1. Official Ethics Approval Letter.....60

List of Tables

1 Insights Matrix 27

1. Introduction

1.1 Background and Motivation

Software has become a key component of modern infrastructure and a cornerstone for many services, ranging from financial systems to healthcare, education and entertainment. As development for these services has accelerated, the derived environmental costs of maintaining these digital ecosystems have scaled upwards tremendously. Early attempts to make Information and Communication Technology (ICT) more sustainable have always focused more on hardware efficiency, such as reducing power consumption, limiting e-waste generated or enhancing manufacturing processes. However, as software systems have grown to include cloud and mobile domains, researchers have realised that unsustainable practices in software development and maintenance would have long-term detrimental effects.

Green and sustainable software engineering has emerged as a response to this realisation. This field makes the case that sustainability must be incorporated into every phase of the software development life cycle and not just considered in deployment or disposal. Despite creating academic frameworks like the GREENSOFT model (Naumann et al. 2011), the Green Software Development Model (Shenoy & Eeratta 2011) or the Karlskrona Manifesto for Sustainability Design (Becker et al., 2015) that support this concept, industrial adoption remains low. Many organisations lack reasonable methods of tracking sustainability and instead prioritise their focus on short-term performance metrics.

The motivation for this research was discovered from the existing gap between the theory and practice. While sustainability is often discussed in terms of carbon output or energy consumption, there is less attention on software lifecycle sustainability. This is the viewpoint of how projects are maintained, the state they are in when abandoned and what happens to the time, infrastructure and potential that become unused after development ceases. A sample of this can be observed from the high number of outdated mobile applications or inactive open-source repositories which inspired the link between project abandonment and software sustainability. The objective inspired an exploration into whether abandoned or stagnating software projects are a contributing factor to the unsustainable practices in the software industry.

1.2 Problem Definition

Existing research into sustainability in computing typically frames the problem around measurable physical resources such as power usage, server energy consumption, or storage requirements. While

these are important considerations, they don't encapsulate the full environmental and economic footprint that software induces. The abandonment of software, whether it is mobile applications left on app stores or open-source projects that have ceased development, represents a unique and different form of waste. When a system is created but never updated, reused or fully retired, the human effort, technical infrastructure and opportunities for future impact for that system become effectively lost.

In digital ecosystems where there are a multitude of applications or repositories, this pattern creates an expanding layer of digital waste that consumes resources indirectly as a result of never being used, server hosting, and neglected data. The absence of structured sustainability metrics to measure these losses means that they often go unnoticed. Consequently, the main problem that this research will address is not the energy or storage wasted during execution but the time, labour or potential misspent through obsolescence and neglect. Understanding the patterns derived from this could lead to new frameworks that promote proactive maintenance and responsible disposal, thus leading to greater software longevity.

1.3 Scope of Study

This project investigates sustainability in software engineering through two main research streams: large-scale data analysis and perception of industry awareness. The first stream focuses on quantitative evidence gathered through existing software development applications. Using Node.js-based scraping tools, data was collected from three distinct sources: the Google Play Store, the Apple App Store and GitHub repositories. For the app stores, metrics like update frequency, install counts or review counts were used to identify applications that were likely to become inactive or abandoned. For GitHub, repository metadata, such as the last push date and star counts, were used to identify the vitality of large-scale software projects. The second stream examines the human side of software sustainability, particularly surrounding current industry awareness of the problem. This was completed through an anonymous survey targeting software developers and engineers in New Zealand. The survey explored the awareness, challenges and mindsets related to adopting green software practices in industry. This dual-stream method provided a holistic understanding of sustainability issues, blending the qualitative human aspect of surveying with the quantitative data analysis from scraping.

The approach taken was to combine results together to produce an Insights Matrix, which is a framework designed to highlight the connections between abandonment indicators, sustainability awareness and possible areas of improvement. As a solution, this matrix does not produce carbon or

energy data and instead focuses on actionable indicators aimed at the benefits of software and its long-term usage. This approach keeps the focus on digital sustainability rather than traditional hardware-related factors, which makes it easier for researchers to consider its usage in practical settings.

2. Background and Literature Review

This chapter provides a condensed summary of the full literature review conducted for this study, which is presented in Appendix A. It outlines the theoretical foundations that inform this research, beginning with an overview of sustainability in computing and how it has changed as a result of green software engineering. This chapter explains how sustainability integrates with the Software Development Life Cycle (SDLC) and discusses the consequences regarding software abandonment. This will lead into understanding the barriers preventing widespread adoption of sustainable practices in industry. Together these sections establish the academic context required to justify how sustainability and software engineering are inextricably linked.

2.1 Sustainability in Computing

Early discussions involving sustainability in Information and Communication Technology (ICT) have always been hardware-focused. With the growing demand for software in society, concerns have been raised for energy efficiency in servers, e-waste for obsolete devices and resource-intensive electronics manufacturing (Naumann et al., 2011; Bekaroo et al., 2013). While advancements have been made by utilising initiatives like Energy Star certifications or eco-labelling to encourage improved hardware efficiencies, the environmental footprint of software is often overlooked (Mayr et al., 2016). As large-scale computer systems scale in complexity, the resource need for software becomes more evident, and this is particularly noticeable in software applications from artificial intelligence to enterprise cloud platforms constantly consuming computation resources (Vartziotis et al., 2024; Eeckhout, 2025). Additionally, software-driven hardware obsolescence contributes to forcing inefficient but still functional devices into retirement, adding to global e-waste (Hilty et al., 2006). It is quite clear from this that sustainability in ICT cannot be viewed strictly as a hardware problem. Both first-order impacts and systemic effects must be addressed, and this establishes the need for a holistic approach where software is specifically made to be sustained longer to reduce future impacts (Becker et al., 2015).

2.2 Foundations of Green Software Engineering

As a response to the growing recognition for software's environmental impact, the field of Green and Sustainable Software Development was created (Dick & Naumann, 2010; Naumann et al., 2011). From this we can divide "green software" into products designed to minimise environmental harm and "green software engineering" into solutions where sustainability practices are embedded in all steps of the development process from creation to end-of-life (Mahmoud & Ahmad, 2013). This distinction is important, as one focuses on the outcome while the other focuses on the

procedure. A popular conceptual framework tied to this distinction is the terms “Green by IT”, related more to green software, and “Green in IT”, relating to green software engineering (Naumann et al., 2011; Dick & Naumann, 2010).

Several models have been introduced that supplement these ideas. The GREENSOFT Model embeds sustainability throughout the entire software development lifecycle (Naumann et al., 2011). The Green Software Development Model highlights the usage of energy-aware practices during programming, testing and deployment (Shenoy & Eeratta, 2011). The Karlskrona Manifesto frames sustainability as a fundamental attribute alongside security and usability (Becker et al., 2015). Despite these numerous frameworks, there is yet to exist a defined, universally accepted metric that includes the lifecycle carbon footprint, energy efficiency in consumption and hardware longevity (Venters et al., 2014; Albertao et al., 2010).

2.3 Software Lifecycle and Abandonment

Sustainability considerations are considered essential when viewed in conjunction with the Software Development Lifecycle (SDLC). By incorporating sustainability into the requirements and design stages, such as supporting backwards compatibility or minimising update frequency. During development and testing, practices such as energy-aware coding and using monitoring tooling like Joulemeter or Greenalytics help identify inefficiencies (Kansal et al., 2010; Zapico et al., 2010). Deployment and maintenance stages also introduce challenges in sustainability, such as bloated update cycles or inefficient hosting environments (Mahmoud & Ahmad, 2013). These can be mitigated, respectively, by reducing installation package sizes and using centralised/cloud-based deployments. Finally, the disposal stage, often overlooked from the software perspective, can invoke sustainable change by planning for decommissioning systems or by extending hardware lifespans by optimising software (Naumann et al., 2011).

The consequences that stem from neglecting sustainability are evident in cases of large-scale project abandonment. Projects such as the FBI Virtual Case File, the UK e-Borders programme, and the Taurus system all collapsed due to poor management, scope creep, overestimating requirements, and technical overstretch (Marchewka, 2014; Alami, 2016; Drummond, 1999). These failures wasted not only billions in financial resources but also created significant environmental costs that were never mitigated (Venters et al., 2014). Some of these costs were electricity consumed during development, discarded infrastructure, or premature hardware obsolescence. By incorporating sustainability principles such as modular design, lifecycle metrics or goal-based decision-making, many of these failures would not have occurred.

2.4 Adoption Challenges in Industry

Despite the growing demand for software systems in industry and the recognition of the environmental impact imposed by it, the adoption of green software practices remains very limited. One of the primary barriers is financial benefits (Mahmoud & Ahmad, 2013; Valmohammadi & Mortaz Hejri, 2023). Organisations will typically prioritise the short-term financial performance over the long-term sustainability benefits that these practices would provide. Traditional return-on-investment frameworks aren't designed to calculate the benefits that greener practices would employ (Kern, 2017). Practices like reducing energy consumption, extending hardware lifecycles or preparing companies for future regulatory compliance are difficult to quantify. As a result, sustainability is often treated as an optional or secondary concern during planning and development. This is further complicated by modern development structures such as agile sprints or continuous integration/continuous development pipelines. Organisational pressures to ensure costs are minimised and ensure fast product release are another factor which reduce adoption, as retraining staff, rethinking infrastructure and delaying deployment would be necessary. These constraints make it unlikely for companies to embed sustainability into their roadmap unless clearer incentives and measurement standards are introduced to drive the change (Naumann et al., 2011; Becker et al., 2015).

Beyond just economics, there are also significant awareness and social barriers that delay industry adoption. Many developers, project managers, and business executives remain unaware of the sustainability implications entangled between software and the environment, viewing it as trivial compared to the more visible hardware (Deng & Ji, 2015; Kern, 2017). Available tools for measuring sustainability, such as Joulemeter, Greenalytics, or the Green Power Indicator, remain underdeveloped and rarely used in mainstream work environments (Kansal et al., 2010; Zapico et al., 2010). Additionally, while awareness can exist, misconceptions can lead to opposition, resulting in cases where sustainability is believed to be a trade-off that reduces performance or usability in exchange (Becker et al., 2015). In work environments, where speed is the priority, teams that use agile or lean methodologies will typically view sustainability to be an impediment to productivity rather than an attribute equal to reliability, scalability or security. Real-world projects often require trade-offs to be made, as deadlines, budgets and client expectations will outweigh environmental goals (Venters et al., 2014). These challenges are an example of some of the limitations that make academic frameworks such as the GREENSOFT model or the Karlskrona Manifesto not widely known, reducing overall adoption in industry. For sustainability to become a key aspect of software

engineering practice, it must be reframed as a core requirement rather than an optional improvement.

3. Methodology

3.1 Research Design

This research adopts a mixed-methods approach, combining quantitative data scraping from public software platforms with the qualitative insights produced from a targeted industry survey. The design for this approach was based on the need to gain an understanding from both the scale of software abandonment and the mentality of current developers associated with it. Quantitative data was gathered from three distinct major digital ecosystems: the Google Play Store, the Apple App Store and GitHub (Android, 2025; Apple, 2025; GitHub, 2022). These sources were chosen to identify trends like application inactivity or update frequency. These data sources provide measurable indicators of software vitality across different domains of practice, mobile applications and open-source repositories.

To complement this technical data, a qualitative anonymous survey was created to capture the perceptions of green software engineering among software engineers and developers working primarily in New Zealand. This dual-method approach ensured that the data gathered would address both the objective evidence of unsustainable patterns in software as well as the subjective understanding of sustainability within the profession.

The research was performed following a specific flow that can be summarised into three stages:

1. Data Acquisition: building scrapers to automate collection of application and repository metadata using Node.js.
2. Survey Deployment: distribution of an online ethics-approved questionnaire to collect opinions from professionals and spread awareness.
3. Data Synthesis: consolidation of dual stream results into an Insights Matrix, highlighting the correlations discovered between sustainability awareness, project inactivity and abandonment indicators.

By using this sequence. It was possible to perform a clear investigation on where sustainability breaks down in practice, both from applications and human mindsets, to identify the broader challenges present in development ecosystems.

3.2 App Store and Repository Scraping

To investigate software abandonment on a large scale, gathering data from public digital storage ecosystems was necessary. To gather this data, the primary approach was to use web scraping

technologies to gather measurable indicators of software from different development environments. This approach allowed for the analysis of multiple different applications and repositories without requiring direct developer involvement, thus providing a realistic, unbiased view of how frequently software is updated or neglected. Before any data collection could be undertaken, research was done to identify how each platform manages concepts related to app inactivity and removal to help identify factors that would be suitable for the criteria of applications that get scraped. Additionally, several candidate scraping methods and libraries were reviewed to determine the most appropriate and ethical tools for automated data extraction. The analysis derived from this research was focused on deciding the consistent criteria that could be used across platforms.

3.2.1 Google Play Store

Research into the Google Play Store's policies revealed that app visibility and availability are closely tied to update frequency and target Application Programming Interface (API) levels (Android, 2025). According to Google's most recent update, effective August 2025, all existing applications must target Android 14 or higher to remain discoverable, while new applications must target Android 15 or later. Apps that fail to meet these basic requirements lose the right to remain visible on the Play Store (Android, 2025). This system enforces a rolling lifecycle of compliance where older, unmaintained applications progressively lose visibility and become abandoned. Understanding this relationship between visibility and policy requirements is integral to designing scraping criteria.

Prior to data collection, research was conducted to determine which indicators would be most suitable for identifying app inactivity. Industry and academic sources indicated that update frequency and installation count were the strongest predictors of abandonment. Specifically, applications with fewer than 1,000 installations often received less attention from their developers on account of not being investable enough (Joseph, 2024). On the other hand, applications that had not been updated within a year typically fall out of compliance with Google's visibility standards. These insights formed the conceptual foundation for how inactivity could be observed in a measurable and replicable way before proceeding to data extraction.

3.2.2 Apple App Store

Preliminary research into Apple's application policies reveals a more stringent approach compared to Google's framework. As of April 2025, all iOS and iPadOS applications submitted to the App Store must be built using Xcode 16 with the iOS 18 Software Development Kit (SDK) (Apple, 2025). Updates that don't meet this requirement will be blocked for all applications. Developers of apps that do not receive an update within three years and fall below a minimal download threshold

will get a notice that their application is at "risk of removal". Developers are given 90 days to update their software before it becomes undiscoverable from public search results, although still accessible to existing users.

This research phase shows that an understanding of the rules regarding removal and visibility can define how the platforms enforce sustainability in their ecosystem. Because Apple does not publicly disclose metrics like installation counts to avoid artificial download inflation, a separate proxy metric, review counts, was chosen. This metric, often used in academic analyses of mobile applications, serves to meet the standards for user adoption and engagement, like installation counts. Combined with update frequency, review count provides a balanced way to determine inactivity in Apple's stricter lifecycle model. Together these parameters establish the standardised framework for identifying abandonment in the iOS ecosystem, ensuring that the criterion for scraping is aligned with Apple's compliance structure.

3.2.3 GitHub Repositories

Unlike the closed, policy-driven ecosystem of mobile application stores, GitHub follows an open collaborative structure where repositories have the option to remain accessible indefinitely, regardless of activity level. As such, pre-scraping research was focused on identifying indicators of abandonment for a platform that doesn't impose restrictions for lifecycles or removal. Using GitHub's public documentation and REST API guidelines, the most appropriate metric was the last push date, comparable to update frequency for app stores (GitHub, 2022). As a counter for project vitality, repository popularity metrics like star and fork counts were included.

Repositories that had not been updated in over a year were considered at risk of abandonment, with lower star counts often correlated with reduced community or developer interest. Further examination of GitHub's data structure confirmed that metadata like project description, programming language, licence and archival status could be used to differentiate between software projects from documentation or academic repositories. Establishing this distinction from the beginning, prior to scraping, was crucial, as it ensured only legitimate software repositories focused on meaningful production-orientated projects would be analysed. This provided a reliable foundation for identifying patterns of project neglect and evaluating sustainability on an open-source platform.

3.3 Green Software Adoption Survey

While the app store and repository analysis provides quantitative evidence of software abandonment, it fails to capture the attitude of sustainability currently existing in industry. To

address this, a secondary qualitative method was sought out through an anonymous survey targeting software professionals (Becker et al., 2015). The purpose of this survey was to understand how developers perceive sustainability within their work, the extent to which awareness of green software engineering is present in their skills and the challenges faced in adopting such practices (Venters et al., 2014). This human-centred component complements the technical findings by revealing the gap in awareness and factors that influence how sustainability decisions are applied in real development scenarios.

3.3.1 Ethical Process

Before any survey activity could be conducted, ethical approval was sought through the University of Waikato's Human Research Ethics Committee. This process ensured that all procedures involving participants followed the university's set standards for privacy, transparency and voluntary participation. A formal ethics application (Approval No. 37) was submitted that included an information sheet, consent form and question. Each document explained the rights of participants, the objectives of the research being undertaken and how the data would be secured and anonymised. A copy of the official ethics approval letter is provided in Appendix B.

Respondents of the survey would be informed that their participation was fully voluntary and that they were free to skip questions and that no personal or identifiable information would be collected. As the survey did not collect any names, company affiliations or contact information to avoid potential bias and maintain confidentiality, participants would not be able to withdraw their collected data from the survey post-completion. Approval was granted before survey distribution. Acknowledging that the study met ethical requirements for human research and adhered to data protection expectations as per the university's research policy.

3.3.2 Survey Design

The survey was designed to explore the level of awareness, perception and practical adoption of green software engineering from professional developers and engineers in the software community. To achieve this, the structure involved a combination of text-based, Likert scale, multiple-choice and closed/open-ended questions to gather both measurable and contextual information. The questionnaire was divided into three sections:

1. Ethics Approval – The first section informed the participant of their role in the survey and gained their informed consent. Branching was used here such that if a participant did not wish to provide consent, the survey would end for them at that stage.

2. Background and Demographic Information – The second section gathered general background information on each participant regarding their career title, sector, company size and location.
3. Survey Body – The final section contained the full body of the survey with every question regarding awareness, understanding, and adoption challenges related to software sustainability.

The Likert scale system of questions was used to measure levels of awareness and attitudes numerically, while open-response questions would encourage professionals to comment on personal and organisational experiences. The length of time required to complete the survey was 10 minutes, as this would encourage completion and capture sufficient detail to support the analysis. Its distribution was targeted towards practising software developers and engineers, professionals most likely to encounter sustainability in the SDLC, within New Zealand to gather relevant insight into how sustainability is dealt with at the industry level.

3.3.3 Data Collection

The survey was hosted on Microsoft Forms and distributed digitally through professional and academic channels, including LinkedIn and university pre-approved mailing lists. This approach would ensure that there was no bias in selecting participants for the study. This online format would ensure accessibility and allow responses to be anonymised and safely secured. Data collection was conducted over a five-week period and remained open long enough to gather a diverse range of participants across varying experience levels and professional backgrounds. The online approach was chosen to maximise reach and ensure anonymity was maintained.

4. Implementation

4.1 Technology Stack

The implementation of this research was supported by a variety of open-source, lightweight, accessible sets of tools that prioritised reproducibility and transparency over complexity. The primary environment used was Node.js, chosen for its asynchronous design and extensive background of data-handling libraries. Node.js provided the flexibility to create modular scraping scripts that could handle multiple queries concurrently without breaching rate limits or overwhelming source platforms.

Several packages were integrated to streamline the process. The google-play-scraper and app-store-scraper libraries (Olano, 2015; Olano, 2016) were used to collect mobile application data, while p-limit was used to handle concurrency, and csv-writer was included to structure and export raw metadata into tabular datasets. For the open-source analysis, Octokit, GitHub's official REST API client for Node.js, was utilised to extract corresponding repository information, including last push date, programming language and adoption metrics (GitHub, 2022).

Once datasets were compiled, Microsoft Excel was used as the main environment for data management and preprocessing. Excel's internal tabling and filtering tools were used to clean and organise the data retrieved from the app store and repository scrapers as well as survey results. Functions like pivot tables or graph making were allowed for efficient detection of duplicates, organisation of essential data and visualisation of key information such as update frequency or user engagement. All processing was completed locally using existing metadata, ensuring compliance with the ethical and privacy considerations outlined in Chapter 3.

4.2 App Store and Repository Scraping

Implementation for scraping involved transforming the framework established during Chapter 3 into a practical executable process. Each script was designed to operate independently for its respective platform while keeping their specific criteria in mind for inactivity and adoption. Three independent workflows were created for the Google Play Store, Apple App Store and GitHub (Olano, 2015; Olano, 2016; GitHub, 2022), each developed to handle their respective structures, data limits and activity indicators.

All programs were implemented using lightweight open-source Node.js libraries to collect metadata at scale. Concurrency limits were applied through the p-limit package to prevent throttling during bulk requests. Each workflow followed a similar sequence.

1. Initial Collection – Retrieve large amounts of metadata based on adoption-related criteria such as install or review counts.
2. Filtering – A secondary script is used to apply the inactivity threshold as not updated since July 2024, aligning with earlier defined abandonment criteria.
3. Formatting – Export the processed data to CSV for filtering and convert to JSON for analysis and visualisation.

Consistent inactivity and adoption thresholds were applied across all three subsets corresponding to their platform to ensure results would be comparable.

4.2.1 Google Play Store

The Google Play Store scraper was created in Node.js using the google-play-scraper library to automate metadata retrieval from multiple app categories (Olano, 2015). The script operated by iterating a customised list of seed keywords which represented a broad section of the Play Store's ecosystem, ensuring that results were not limited to a single domain or topic. For each query, the scraper collected structured information, including the application ID, title, developer, category, score, ratings, install count, update date, version, and price type. To ensure rate-limiting was not exceeded, the p-limit package was added to manage concurrency and ensure data collection was smooth. The retrieved data was exported into a CSV file using the csv-writer library for further filtering based on inactivity metrics before being converted to JSON for later analysis.

4.2.2 Apple App Store

The Apple App Store scraper was developed in Node.js using the app-store-scraper library, designed by the same developer as the Play Store module in structure and output (Olano, 2016). Like the Android process, the script iterated through various categories to capture a diverse range of applications. Since Apple does not publicly reveal installation counts, review counts were used as a replacement for the adoption metric of installation count.

Each retrieved application entry contained metadata such as app ID, title, developer name, genre, review count, rating, update date, version, and application URL. The p-limit package was used again to manage concurrency and avoid Apple's request limit thresholds. The extracted data collected after initial scraping targeted the first indicator of less than 20 reviews and was then exported to CSV. After this, the second filtering script was used for gathering applications not updated since July 2024 and finally converted to JSON for analysis and cross-platform comparison.

4.2.3 GitHub Repositories

For open-source repositories, a Node.js script was developed to collect metadata from a manually curated list of repository URLs. This list was assembled by searching GitHub using the query filter to locate projects that had not been updated since before July 2024. Each result was manually reviewed to confirm that it represented a genuine software project rather than documentation, academic material, or archived content. From this process, 25 suitable repositories were selected for analysis.

The script used the GitHub REST API to extract metadata directly from a verified list of repositories (GitHub, 2022). It parsed each repository URL to extract structured fields such as repository title, description, stars, forks, open issues, watchers, size, primary language, licence, creation date, and last push date. The retrieved data was exported directly to JSON such that it would be consistent with the app store scraping and ready for further analysis.

4.3 Green Software Adoption Survey

The survey complemented the scraping data by giving qualitative information on how awareness and adoption of green software engineering practices are in the industry. The goal was to find out how software developers and engineers view sustainability principles in their work and identify barriers preventing it from being used in standard development processes. This section details how the survey was created and managed during execution.

4.3.1 Survey Execution

Microsoft Forms was used to host the survey because of its accessibility, recognition as a popular service, data exportability into Microsoft Excel and ability to enforce branching when necessary. Prior to opening the survey for the target audience, it was carefully organised to ensure that branching, question management and mandatory fields were working correctly. The final version of the survey featured 23 questions divided into three logical sections:

1. Ethical Consent – the first and only branching question ensured that all participants who agreed to the consent statement could continue to participate. Refusal automatically submitted the survey with no information collected.
2. Demographics – questions 2-6 collected non-identifying background information to contextualise later responses. These included company type, role title, company size, whether it was New Zealand-based and whether it was a subsidiary of an international organisation.

3. Main Survey Body – questions 7-23 explored participants' awareness and knowledge of green software engineering. A mixture of formats was used, including:
    a. Awareness and Understanding: questions such as "Do you know what Green and Sustainable Software is?" followed by an open text field for elaboration.
    b. Organisational Practice: Multiple-choice questions on topics related to sustainability in the SDLC and its use in their organisational structure.
    c. Barriers and Motivation: open-ended questions identifying key challenges restricting sustainability from being a key aspect of software development and what would encourage greater adoption.
    d. Interest in Learning: concluding questions that assessed willingness to learn more about sustainable software engineering.

Response validation was utilised to ensure numeric and Likert inputs were correct. The survey remained accessible for five weeks, and during this time, numerous reminders were shared periodically via LinkedIn and professional mailing lists to encourage participation.

4.3.2 Data Handling and Analysis

Once closed, all responses were automatically compiled and displayed within Microsoft Forms, generating built-in summary graphs, percentage distributions and word clouds for open-ended responses. These visualisations gave an immediate overview of participant demographics, awareness levels and main themes without requiring the use of external processing tools.

To complement these summaries, the dataset was exported to Microsoft Excel for additional organisation and comparison. Pivot tables were used to create custom graphs better suited for understanding the comparison against participants. This approach enabled a clearer interpretation of quantitative patterns while retaining the original word cloud insights for qualitative trends.

5. Results

This chapter presents the results obtained from the three digital platforms, Google Play Store, Apple App Store, and GitHub, followed by the findings from the Green Software Adoption Survey. The same inactivity threshold of no updates since July 2024 and equivalent adoption measures (less than 1,000 installs or 20 reviews) were used to ensure consistent comparison.

5.1 Play Store Results

From an initial sample of 816 applications retrieved using the google-play-scraper library, 76 applications were found that met both inactivity and adoption criteria of less than 1000 installs and not updated since July 2024. This represents roughly 9% of the scraped sample. The resulting drop in applications after filtering was expected, as the Play Store's algorithm is designed to promote recently updated or popular apps, making our target group of older, low-engagement users harder to extract.

The metadata collected showed that the most active update periods were concentrated between 2020 and 2024, after which there was a noticeable decline. This pattern aligns with rolling API level enforcement policies, regulating and delisting older applications from search visibility unless they are updated to meet newer Android versions. Categorically the filtered results included a broad range of productivity tools to entertainment apps and games. The overall findings suggest that small-scale or underperforming applications are highly vulnerable to abandonment once user traction stops and that platform-driven compliance requirements accelerate this process.

5.2 App Store Results

The Apple App Store scraper produced a smaller initial dataset of 192 applications that had less than 20 reviews, reflecting the stricter visibility and SDK requirements within Apple's ecosystem. After applying the inactivity filter of no updates before July 2024, 98 applications remained, equating to approximately 51% of the initial sample. Because Apple does not publish installation counts, review counts served as a practical proxy for adoption.

Filtered applications ranged from 2014 to 2020, with a surge in update frequency around 2022. The dataset shows an existing pattern for infrequent updates among low-reviewed applications, many of which were last updated several years prior to the inactivity filter yet remain on the app store. This highlights how Apple's lifecycle enforcement prioritises quality and security over sustainability. Applications that remain compliant but inactive continue to occupy storage space despite a dropping user base until the developer eventually waits for their delisting.

5.3 GitHub Repository Results

Manual selection and validation identified 25 repositories that had not received commits or pushes since before July 2024. Each was confirmed to be an active software project at some stage rather than documentation or archived material. The resulting dataset represented a range of project sizes and types, from small independent utilities to large Software as a Service platforms. Many of these repositories had not been updated in over a year, indicating reduced contribution activity or complete abandonment. The analysis confirmed that while some repositories remained accessible for historical or reference purposes, most showed minimal engagement, few recent commits, and no signs of continued maintenance.

Unlike the mobile application stores governed by policy enforcement, GitHub imposes no restrictions on removal or visibility on inactive projects. This open retention model supports potential reuse but simultaneously leads to long-term accumulation of unmaintained repositories that remain indefinitely. As a result, GitHub's openness highlights a sustainability issue that consumes resources without ongoing development value.

5.4 Survey Results

A total of 17 responses were collected over a five-week period from industry professionals working in software-related organisations, mainly within New Zealand. The survey was designed to capture current awareness, perception, and adoption regarding green software engineering practices, to help the quantitative data gathered through the scraping analysis.

From the demographics section of the survey, 35% of participants identified themselves as software engineers, with the remaining roles ranging from senior developers to technical leads or managers. Over 82% of respondents worked in New Zealand-based organisations employed by small- to medium-sized companies ranging from 11 to 50 employees. This demographic spread provided a balanced snapshot of regional industry perspectives rather than a large international impact.

During the main body section, when participants were asked if they knew about Green and Sustainable Software, 76% of respondents answered "No". Open-ended responses showed that while some participants associated the concept with "energy efficiency" or "environmental impact", few understood the connection to the SDLC. When asked a closed question on familiar concepts, 53% of respondents acknowledged knowing energy-efficient software practices, indicating that current knowledge is limited to certain aspects and not the full breadth of sustainability. This demonstrated that sustainability within software engineering remains largely unrecognised from industry perspectives.

When participants were asked whether their respective organisations considered sustainability during software development or procurement, the most common answer was “Sometimes” at 29%, indicating that professionals’ sustainability was considered but not guaranteed. Alternative resulting answers for “Yes”, “No” and “Not Sure” were split evenly at 24%, confirming that sustainability is an optional addition in the minds of developers. Further questions revealed that 41% of respondents don’t consider sustainability at all in the SDLC, but of the 59% that do, they primarily consider it in the design and development stages, with limited focus for later stages like maintenance or disposal. Similarly, when considering sustainability in other sectors of their organisation, most developers will contemplate its usage in software development or cloud infrastructure, disregarding other areas like network support, cybersecurity or IT consulting. 35% of respondents admitted to not knowing if their own organisation’s technical infrastructure includes sustainable measures in its construction, with 24% indicating that no measures were in place and 29% indicating some measures. This uncertainty suggests that many professionals probably don’t have the authority to know infrastructure-level decisions or sustainability goals of their organisation.

When asked to rate how important inclusion of sustainability practices was in software development, most participants placed it as unimportant, with a split 58.8% choosing either “Unimportant” or “Slightly Unimportant”. This result shows that sustainability is not considered a high-value attribute among software teams. Questioning on how difficult integrating sustainability into existing development revealed that 52.9% chose a neutral position, suggesting uncertainty over opposition. Only a small number found it difficult or very difficult, indicating that while developers don’t actively prioritise sustainable development, they aren’t opposed to its inclusion or see it as a major technical barrier.

While identifying the largest challenges to adopting sustainable software practices, a majority of respondents selected lack of training and awareness, with a minority choosing difficulty in measuring the impact. These two barriers were referenced in both multiple-choice and open-ended word questions, highlighting that informational and measurement gaps remain the highest blockade preventing industry adoption. This was also corroborated in another question for what would motivate organisations to adopt green software engineering practices more effectively, with 24% advocating for awareness and training initiatives. Several respondents also indicated that having clearer benefits and measurable indicators would help justify sustainability inclusion.

Currently, 94% of respondents reported that their organisation does not have a dedicated person or team responsible for managing sustainability initiatives in their workplace. This level of absence reinforces that software sustainability is currently an optional inclusion or informal concern that

most organisations wouldn't consider normally. 76% of participants also claimed to not have access to any tools for detecting or measuring software sustainability, indicating that current tooling is a rarity. When asked if their developers or other IT staff receive training related to sustainable software practices, a majority (59%) replied, "No, not interested," proving a desire to learn exists. 35% replied "Not Sure"; this, combined with the earlier desire, suggests a willingness to learn to develop skills if adequate training and knowledge are provided with an understanding of clear benefits.

In questions related to perceived value, 29.4% of respondents rated green software as "Slightly Valuable" to their company's values, suggesting that it is recognised but not prioritised as a core organisation principle. It can be assumed that software sustainability isn't high enough of a concern for companies to actively include it. When asked whether green software is becoming more prevalent in industry, 41% were unsure, with the remaining answers split between "Yes" or "No". This uncertainty suggests that the field is too unclear, and not enough developers have clear insights into the public representations of software engineering.

Lastly, when participants were questioned on whether they would be more interested in learning about sustainable software engineering practices, 76% of respondents reported "Yes". This strong majority indicates that while current awareness and formal training are lacking, there is a clear passion for technical development in this area.

In summary, the survey results reveal a pattern of low awareness but high interest. While sustainability is absent in many areas of the SDLC, the data indicates that industry professionals are open to incorporating these practices should they be provided with appropriate education, clear financial impacts and measurable frameworks. Together these findings confirm that hurdles to adoption are mainly organisational and informational rather than technical, matching challenges observed in scraping results.

6. Discussion

6.1 Insights Matrix and Comparative Analysis

The Insights Matrix was designed to integrate the quantitative evidence of software abandonment from digital platforms and qualitative insights from professional perceptions into a single comparative framework. Both streams provided a unique perspective: the scraping analysis presented measurable inactivity, while the survey provided the human perceptions for it.

This framework draws on the dual methodology established in Chapter 3. The purpose of this matrix is to connect observed digital behaviour such as inactivity, retention and obsolescence with human and organisational drivers such as awareness, motivation and governance, to identify where sustainable software engineering fails in practices. Synthesised discernments gathered from this Insights Matrix will provide an evidence-based view of the current gaps and opportunities that can help influence future green software engineering decisions.

Aspect	Literature Review	Scraping (Play Store. App Store, GitHub)	Survey (Developers, NZ Industry)	Synthesised Insights
Awareness & Knowledge	Existing frameworks emphasise awareness as the foundation of sustainability, requiring addition in all SDLC stages. Studies indicate most professional are unaware of software sustainability	Abandonment data shows no evidence of sustainable thinking. Applications and repositories are left inactive without structures in place for maintenance or planned retirement.	76% of respondents were unaware of green software with most associating it with energy efficiency rather than the full lifecycle impact.	A persistent gap exists of awareness. While academic models that advocate for sustainability, yet industry adoption is limited to surface level considerations. Bridging this divide requires active education and distribution of existing frameworks

Organisational Adoption	Literature advocates for integrating sustainability into industry structure and project management. Sustainability should be treated as a quality attribute equal to security and usability,	Across ecosystems, abandoned applications and repositories show absence of ownership. Projects are left online but unmanaged.	94% reported no structure for enabling sustainability initiatives. 29% said sustainability is "sometimes" considered in projects.	Organisational abandonment reflects ecosystem neglect. Without a defined individual or group in charge for accountability, sustainability remains theoretical and optional. Aligning with literature means to alter mindsets for sustainability roles and lifecycle policies.
Measurement & Tooling	Studies highlight lack of standard metrics and tools for assessing sustainability in software. Existing tools such as Greenalytics and Joulemeter aren't standard and experimental (Kansal et al., 2010; Zapico et al., 2010).	Scraping identified only basic activity indicators such as installs, reviews and stars. None of these platforms showed lifecycle or sustainability metrics.	76% said their organisation lacked sustainability tools. 59% wanted training in measurement and evaluation	A global measurement blind spot exists. Both practitioners and platforms don't measure and test sustainability. Literature identifies this as a barrier to mainstream adoption, with scraping evidence supporting lack of metrics existing.
Policy & Governance	The Karlskrona Manifesto calls for policies and standards guiding	Platform policies such as Google API levels or Apple SDK	Developers rarely referenced policy as an	Platform and organisational policies prioritise compliance and

	sustainable software however adoption frameworks are still seen as optional (Becker et al., 2015).	versions enforce compliance and security at the compromise of sustainability. GitHub's open model lacks governance entirely resulting in accumulating data without value.	influence with most unaware of regulations promoting sustainable software.	quality over sustainability. Governance frameworks remain reactive but not preventative which reduces systemic regulations.
Value & Motivation	Literature notes that sustainability is undervalues compared to attributes because benefits aren't clear or long-term. Incentives should link sustainability to business performance.	Abandoned projects typically have low engagement or commercial value causing them to stop being used. Applications will be deprioritised by developers if community traction drops.	58.8% rate the value of sustainability as unimportant or slightly unimportant. Lack of measurable benefits and awareness are major hurdles of sustainability.	The industry's value system is highly dependent on speed, visibility and delivery. Individuals and organisations will prefer short term benefits over longevity. This cultural trend undermines systemic mindset encouraged in academic frameworks.
Future Interest & Education	Research advocates embedding sustainability topics in tertiary curriculums and	No evidence of reflection or reuse exists among inactive projects. This suggests that	76% of developers expressed willingness to learn more about	Enthusiasm for training indicates strong potential for cultural change if sustainability is introduced through

	employing continuous professional training in industry.	potential for learning exists to reduce abandonment.	sustainability. 59% wanted structured formal training.	education and certification; supporting literature emphasises education as a long-term solution.

Table 1 – Insights Matrix

The Insights Matrix presented in Table 1 reveals how sustainability failures in software are multi-layered, spanning educational, organisational and technological sectors. These layers collectively manipulate how sustainability is understood and implemented in industry. When compared against theoretical principles established in literature, results indicate that transferring from awareness to action remains limited given the systemic nature of the challenge.

At the educational and awareness level, the analysis indicates that while theoretical models such as GREENSOFT (Naumann et al., 2011) and the Karlskrona Manifesto (Becker et al., 2015) promote embedding sustainability across the entire SDLC, including it in practice is quite difficult. Most surveyed professionals are unaware of these concepts, and scraping data revealed no evidence of lifecycle management related to sustainability. This shows that awareness and implementation are both theoretical constructs as of this point in time.

At the organisation level, both the survey and digital datasets highlight a lack of responsibility for ensuring software longevity. Academic literature points out that governance and accountability are necessary for sustainable development, yet 94% of developers don't have any such frameworks built into their organisational structure to implement these changes. The persistence of obsolete software observed across all platforms reinforces this issue that sustainability is rarely considered in management processes.

The absence of quantifiable metrics is an ever-present issue. Neither industry professionals nor digital ecosystems have metrics in place that allow sustainability to be measurable beyond surface indicators such as updates or installations. This aligns with the literature's recognition that the absence of measurable indicators hinders adoption and hurdles progress.

Despite these deficiencies, findings show that an opportunity for intervention through education and training exists. There exists strong expressed intent for learning about sustainable practices corresponding to academic reports to embed these principles in existing professional development

and tertiary education. Addressing the knowledge and measurement gaps through a structured education system could provide a strong foundation for future long-term change.

In summary, the comparative analysis confirms that sustainability theory and practice have an existing significant gap. Academic frameworks provide a conceptual basis, but awareness, governance and measurement mechanisms are not yet implemented and are absent across industry development and digital platforms. Coordinated approaches that tackle education, organisational and policy-driven issues must be taken to ensure that sustainability is transformed from a conceptual understanding to an operational practice.

6.2 Key Challenges in Sustainable Software Engineering Adoption

Findings from the Insights Matrix in Table 1 highlight a set of reoccurring challenges that restrict the advancement of sustainable software engineering. These challenges occur across various levels, preventing sustainability from being embedded into software practice. These challenges are systemic in nature, making them difficult to incorporate into existing software infrastructure.

The first and foremost challenge is awareness and understanding. Despite extensive academic literature promoting sustainability in all stages of the SDLC, many professionals don't have any knowledge of these concepts, often reinterpreting them as energy efficiency rather than lifecycle responsibility. This lack of conceptual understanding is a significant limitation stopping practical implementation of green software engineering.

A second challenge is presented in organisational accountability. Findings suggest that sustainability is rarely incorporated into management structures or policy frameworks, and most organisations will not have a formal role dedication to ensuring sustainable software initiatives are in place. This absence of ownership is reflected in digital platforms where inactive projects persist without a structured plan for retirement or reuse.

The third challenge is measurement. Both academic and industrial contexts reveal that a lack of standardised indicators is in place for addressing sustainability outcomes. Current evaluation remains limited to superficial metrics such as updates or installation counts, preventing sustainability from being effectively monitored and quantified.

Lastly, while there is clear evidence for education and interest in learning, pathways to induce training and professional development are limited. Survey responses show that developers are open to learning about sustainability, yet not many structured resources exist or incentives are available to motivate and support this ambition.

Overall, these challenges demonstrate how the gap between theory and practice remains primarily due to weak institutional structures and limitations on awareness rather than technical barriers. Addressing these detriments through education and organisational governance while establishing measurable indicators is critical for integrating sustainability as a core attribute of software engineering practice.

7. Conclusion

This research investigated sustainability as a software engineering problem by observing how technical and human perspectives influence how software projects are maintained, abandoned and understood in practice. Through a mixed-methods approach of combining large-scale scraping on digital platforms with a survey for software professionals, this study identified measurable evidence for software inactivity and how a lack of industry awareness limits sustainable software engineering principles.

The findings confirm that sustainability failures are present in software and they are layered across educational, organisational, and technological contexts. While frameworks exist to educate and provide comprehensive guidance theoretically, they are not yet effectively present in professional and industrial practice. Industry awareness remains limited, with organisational accountability practically absent and existing methods for evaluation failing to capture sustainability beyond surface metrics.

Despite these limitations, the research shows that the potential for progress is real. Survey responses reveal that professionals are willing to learn and apply sustainable practices, indicating the challenge is not in human resistance but in the lack of structured education, tools and benefits. With the establishment of measurable indicators, the software development sector can transform sustainability from a theory into an operational reality.

Overall, these results demonstrate that the primary hurdles hindering sustainable software engineering are systemic rather than technical. Addressing them will require organised tactics that combine education, measurement and policies to ensure that sustainability becomes a directing part of future software engineering practice.

7.1 Future Work

Future work should focus on expanding both datasets for digital ecosystems and survey responses to strengthen the evidence base for software abandonment and human perspectives, respectively. Developing measurable sustainability indicators and lightweight tooling to calculate lifecycle activity would allow a greater impact for green software engineering to be seen. Further studies could explore the long-term effects of organisational policy and education by integrating aspects of sustainability principles into tertiary education and professional training. Finally, refining the Insights Matrix into a practical framework that can provide a set of recommendations, helping developers and decision makers to understand, evaluate and improve their software applications more systematically.

References

Alami, A. (2016). The UK e-Borders Project Failure. In PM World Journal The UK e-Borders Project Failure. https://pmworldlibrary.net/wp-content/uploads/2016/03/pmwj44-Mar2016-Alamiuk-eborders-project-failure-featured-paper.pdf

Albertao, F., Xiao, J., Tian, C., Lu, Y., Zhang, K. Q., & Liu, C. (2010). Measuring the Sustainability Performance of Software Projects. 2010 IEEE 7th International Conference on E-Business Engineering, 369–373. https://doi.org/10.1109/icebe.2010.26

Android. (2025, August 31). Meet Google Play's target API level requirement. Retrieved from Android Developers website: https://developer.android.com/google/play/requirements/target-sdk

Apple. (2025, April 24). SDK minimum requirements - Upcoming Requirements - Apple Developer. Retrieved from Apple.com website: https://developer.apple.com/news/upcomingrequirements/?id=02212025a

Becker, C., Chitchyan, R., Duboc, L., Easterbrook, S., Penzenstadler, B., Seyff, N., & Venters, C. C. (2015). Sustainability Design and Software: The Karlskrona Manifesto. 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering. https://doi.org/10.1109/icse.2015.179

Bekaroo, G., Bokhoree, C., & Pattinson, C. (2013). Towards Green IT Organisations: A Framework for Energy Consumption and Reduction. ResearchGate, 8(3). https://doi.org/10.18848/1832-3669/CGP/v08i03/56290

Deng, Q., & Ji, S. (2015). Organizational Green IT Adoption: Concept and Evidence. Sustainability, 7(12), 16737–16755. https://doi.org/10.3390/su71215843

Dick, M., & Naumann, S. (2010). Enhancing Software Engineering Processes towards Sustainable Software Product Design (pp. 706–715). EnviroInfo. https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=98d04de0318b252f273e3cf36cc385253542b924

Drummond, H. (1999). Are we any closer to the end? Escalation and the case of Taurus. International Journal of Project Management, 17(1), 11–16. https://doi.org/10.1016/s0263-7863(97)00074-4

GitHub. (2022, November 28). GitHub REST API. Retrieved from GitHub Docs website: https://docs.github.com/en/rest?apiVersion=2022-11-28

Hilty, L. M., Köhler, A., von Schéele, F., Zah, R., & Ruddy, T. F. (2006). Rebound effects of progress in information technology. Poiesis & Praxis, 4(1), 19–38. https://doi.org/10.1007/s10202-005-0011-2

Joseph, S. (2024, September 8). The difference between apps that win and fail. Medium. https://medium.com/beyond-agile-leadership/the-difference-between-apps-that-win-and-failb4f4bb647b61

Kansal, A., Zhao, F., Liu, J., Kothari, N., & Bhattacharya, A. (2010). Virtual machine power metering and provisioning. Symposium on Cloud Computing. https://doi.org/10.1145/1807128.1807136

Kern, E. (2017). Green Computing, Green Software, and Its Characteristics: Awareness, Rating, Challenges. Progress in IS, 263–273. https://doi.org/10.1007/978-3-319-65687-8_23

Mahmoud, S. S., & Ahmad, I. (2013). A green model for sustainable software engineering. International Journal of Software Engineering and Its Applications, 7(4), 55–74. https://www.researchgate.net/publication/285762532_A_green_model_for_sustainable_software_engineering

Marchewka, J. T. (2014). The FBI Virtual Case File: A Case Study. Communications of the IIMA, 10(2). https://doi.org/10.58729/1941-6687.1132

Mayr, H. C., Pinzger, M., & Kern, E. (2016). The Development of an Eco-Label for Software Products - a Transdisciplinary Process? (p. 1285). Gesellschaft für Informatik. https://cs.emis.de/LNI/Proceedings/Proceedings259/1285.pdf

Naumann, S., Dick, M., Kern, E., & Johann, T. (2011). The GREENSOFT Model: A reference model for green and sustainable software and its engineering. Sustainable Computing: Informatics and Systems, 1(4), 294–304. https://doi.org/10.1016/j.suscom.2011.06.004

Olano, F. (2015). GitHub - facundoolano/google-play-scraper: Node.js scraper to get data from Google Play. Retrieved from GitHub website: https://github.com/facundoolano/google-playscraper?tab=readme-ov-file

Olano, F. (2016). GitHub - facundoolano/app-store-scraper: scrape data from the itunes app store. Retrieved from GitHub website: https://github.com/facundoolano/app-store-scraper

Shenoy, Sanath. S., & Eeratta, R. (2011). Green software development model: An approach towards sustainable software development. 2011 Annual IEEE India Conference. https://doi.org/10.1109/indcon.2011.6139638

Valmohammadi, C., & Mortaz Hejri, F. (2023). Designing a conceptual green process model in software development: A mixed method approach. International Journal of Information Management Data Insights, 3(2), 100204. https://doi.org/10.1016/j.jjimei.2023.100204

Vartziotis, T., Dellatolas, I., Dasoulas, G., Schmidt, M., Schneider, F., Hoffmann, T., Kotsopoulos, S., & Keckeisen, M. (2024). Learn to Code Sustainably: An Empirical Study on Green Code Generation. Proceedings of the 1st International Workshop on Large Language Models for Code, 30–37. https://doi.org/10.1145/3643795.3648394

Venters, C. C., Lau, L., Griffiths, M. K., Holmes, V., Ward, R. R., Jay, C., Dibsdale, C. E., & Xu, J. (2014). The Blind Men and the Elephant: Towards an Empirical Evaluation Framework for Software Sustainability. Journal of Open Research Software, 2(1). https://doi.org/10.5334/jors.ao

Zapico, J. L., Turpeinen, M., & Brandt, N. (2010). Greenalytics: a tool for mash-up life cycle assessment of websites. ResearchGate. https://www.researchgate.net/publication/229037499_Greenalytics_a_tool_for_mashup_life_cycle_assessment_of_websites

Appendix A

Literature Review

The literature review is included as an appendix on the following page.

ENGEN582-25X

Honours Research and Development Project



THE UNIVERSITY OF
WAIKATO
Te Whare Wānanga o Waikato

Sustainability as a Software Engineering Problem

Aryan Thanki

Judy Bowen

Table of Contents

1. Introduction 40
2. Sustainability in Computing 42
2.1 Traditional Focus on Hardware 42
2.2 The Growing Environmental Burden of Software 42
2.3 Energy Consumption Trends in Modern ICT 43
2.4 System Effects and Rebound Phenomena 43
2.5 Need for a Holistic Sustainability Approach 44
3. Foundations of Green Software Engineering 45
3.1 Defining Green and Sustainable Software Engineering 45
3.2 Green by IT versus Green In IT 45
3.3 Major Frameworks and Models 45
3.4 Challenges in Defining Metrics 47
4. Software Development Life Cycle 48
4.1 Requirements and Design Stages 48
4.2 Development and Testing Stages 49
4.3 Deployment, Maintenance and Disposal Stages 50
4.4 Metrics for Sustainable Software 50
5. Software Abandonment and Large-Scale Failures 52
5.1 Causes of Abandonment 52
5.2 FBI Virtual Case File 52
5.3 UK e-Borders 52
5.4 Taurus 53
5.5 Reason for project failure 53
6. Green Software Adoption in Industry Challenges 54
6.1 Economic and Organisational Constraints 54
6.2 Lack of Awareness and Tooling 54
6.3 Cultural and Technical Misconceptions 55
6.4 Trade-offs and Complexity in Real-World Projects 55
7. Summary 56
References 57

1. Introduction

Information and Communication Technology (ICT) is a broad category that involves many subsets of industry. With the growing demand for large, complex systems comes an equally increasing carbon footprint because of modern software development. While significant efforts have been made to control sustainability at the hardware level over the years, such as developing energy-efficient processors or promoting e-waste management (Naumann et al., 2011), the environmental footprint of software systems has grown significantly and is a growing contributor to global carbon emissions (Becker et al., 2015; Eeckhout, 2025).

Modern software applications, cloud computing infrastructure, and artificial intelligence are just some examples of the immense workload that the energy industry faces to satisfy computational resources. These demands aren't limited to development alone but also persist through deployment, maintenance, and everyday operation (Vartziotis et al., 2024). Moreover, software-induced obsolescence, where evolving software can lead to hardware becoming prematurely unusable, adds unnecessary quantities of e-waste (Hilty et al., 2006). Studies have shown that even as individual device efficiency improves, total energy efficiency does not rise, with consumption becoming greater due to a rebound effect and escalating usage patterns (Hilty et al., 2006; Eeckhout, 2025).

Recognising this, the field of Green and Sustainable Software Engineering was born, with the intent of developing software that is intrinsically sustainable ("Green in IT") but also for designing software that enables sustainability elsewhere (Dick & Naumann, 2010; Naumann et al., 2011). Foundation models have been created, such as the GREENSOFT model (Naumann et al., 2011), the Green Software Development model (Shenoy & Eeratta, 2011), or the Karlskrona Manifesto (Becker et al., 2015), which provide a framework for sustainability embedded throughout the software lifecycle, from requirements gathering all the way to the disposal phase. In conjunction, analysis tools like the Green Tracker (Amsel & Tomlinson, 2010) and Greenalytics (Zapico et al., 2010) for observing energy statistics have been designed to visualise and measure the environmental impacts of software.

Despite these advancements, significant gaps exist between research targets and real-world implementation. Adoption of green software engineering practices in industry remains limited, with barriers such as lack of economic incentives, limited developer awareness, or lack of integration into Agile working environments (Kern, 2017; Deng & Ji, 2015). Furthermore, large-scale project failures such as the FBI Virtual Case File or the UK e-Borders systems (Marchewka, 2014; Alami, 2016a) showcase the economic burdens of unstructured, unsustainable software practices.

This literature review will aim to critically examine the intersection between sustainability and software engineering. It will explore foundational concepts, models, and measurement tools; assess practices around the Software Development Life Cycle (SDLC); and analyse key barriers to adoption in the industry. Investigation of case studies involving large-scale software abandonment will be shown to highlight the real-world consequences of neglecting software development practices.

The key research questions this review will answer are

1. How has the understanding of sustainability developed in software, and what frameworks best capture its evolution?
2. At which stages of the software development life cycle is sustainability most successfully embedded?
3. What are the primary barriers to employing sustainable software development practices, and how can they be overcome?

By synthesising insights from foundational and emerging literature, this review seeks to outline the path forward by increasing knowledge surrounding sustainability as a software engineering problem.

2. Sustainability in Computing

2.1 Traditional Focus on Hardware

Early debates surrounding sustainability in the Information and Communication Technology (ICT) space had a skewed, hardware-centric focus. Environmental concerns were mainly associated with the material aspects of computing, such as the energy consumption of servers, e-waste generated by obsolete devices, or the resource-intensive aspects involved in electronics manufacturing (Naumann et al., 2011). Efforts to address these issues led to advancements in hardware design like energy-efficient processors, improved power management systems, and hardware recycling programmes (Bekaroo et al., 2013). Energy star certifications and other forms of eco-labelling initiatives for devices became common practice, encouraging manufacturers to pursue incremental gains in device manufacturing for energy efficiency (Mayr et al, 2016)

However, this hardware-focused perspective presented a limited view of ICT's full environmental scope. Hardware was often considered to be the primary contributor to carbon emissions, while the role of software was largely ignored. As noted by Mocigemba (2006), focusing on hardware exclusively neglects the system-wide interactions that drive the energy demand. Therefore, while significant improvements had been made, overall ICT emissions continued to rise, and it was determined that additional factors were present. This narrow framing of sustainability as a sole hardware problem delayed critical investment in the lifecycle impacts of software systems. This oversight has had profound consequences, particularly as software has become increasingly complex and widespread across multiple industries (Penzenstadler, 2013).

2.2 The Growing Environmental Burden of Software

As ICT systems evolved, software's contribution to environmental deterioration became increasingly evident. Modern software systems, whether they are intelligence algorithms or large-scale cloud infrastructure, require equally substantial computation resources throughout their lifecycle (Vartziotis et al., 2024). According to Vartziotis et al. (2024), the energy required to train a single large-scale model can be equivalent to the annual electricity consumption for 100 homes, highlighting the immense carbon footprint associated with modern software development.

This environmental drain is further exacerbated by the inclusion of data centre energy consumption. According to Naumann et al. (2011), global energy consumption from data centres increased from 58 billion kilowatt-hours (kWh) in 2000 to 123 billion kWh in 2005. This doubling over five years reflects not only the increasing complexities and resource intensities of software but also the hardware demands. Other studies have shown this unsustainable trend, noting that software-driven services outpace gains made in hardware energy efficiency (Mocigemba, 2006; Eeckhout, 2025). Software-induced hardware obsolescence is another variable added to the problem. As newer software versions demand greater processing power and memory, older hardware becomes inadequate despite remaining fully functional (Hilty et al., 2006).

As noted by Eeckhout (2025), the increasing complexity and ubiquity of software systems imply that their environmental footprint cannot be dismissed as minimal. Instead, software must be recognised as a first-class actor in the sustainability conversation, both as a direct source of environmental impact that requires mitigation and as a potential enabler of greener practices.

2.3 Energy Consumption Trends in Modern ICT

The global need for computational power has expanded rapidly over the last two decades, driven mainly by growth in cloud computing, big data analytics, and artificial intelligence. Despite improvements made in processor efficiency or server design, the aggregate energy demand of digital services has continued to grow (Mocigemba, 2006; Eeckhout, 2025). Data centres now consume an estimated 1-1.3% of total global electricity, which amounts to 240-340 terawatt-hours annually (Vartziotis et al., 2024). Artificial intelligence (AI) workloads represent one of the most energy-driven drivers of this growth. Recent analysis by Eeckhout (2025) identifies that overall carbon footprints continue to grow without substantial intervention. As shown in Figure 1, using the compound annual growth rate (CAGR), more forceful methods of sustainability may be needed to offset increasing usage patterns.

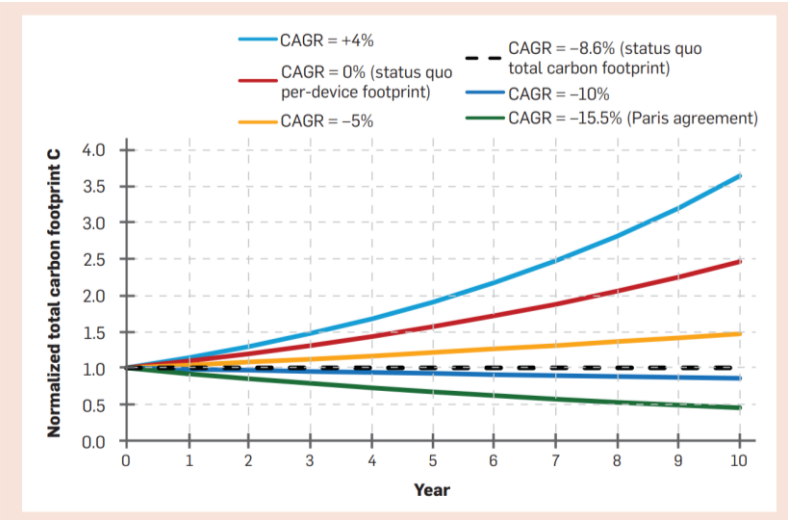


Figure 1 - Total carbon footprint normalized over time for different per-device scaling scenarios (adapted from Eeckhout, 2025)

Training a single large natural language processing model such as OpenAI's GPT-3 reportedly generated over 532 tonnes of CO_2 emissions (Vartziotis et al. 2024; Ceurstemont, 2024). These lifecycle emissions, which include both training and testing phases, often rival or exceed the lifetime emissions count of traditional manufactured goods like automobiles (Ceurstemont, 2024). As applications for AI, blockchain, and big data analytics continue to expand in industry, there is a growing sense to rethink how software engineering practices can be altered beyond hardware optimisations (Becker et al., 2015; Naumann et al., 2011).

2.4 System Effects and Rebound Phenomena

While first-order impacts such as direct energy use and emissions from data centres are increasingly well understood, second and third-order systemic impacts are much more difficult to classify and measure. Second-order effects refer to the indirect consequences of using ICT, such as dematerialisation of services. This could be simplified to how virtual meetings reduce travel or the optimisation of logistics through software (Naumann et al., 2011). Third-order effects involve broader social and economic transformations that evolve over time and produce surprising undesirable environmental outcomes (Hilty et al., 2006). Examples of this include changes in consumer behaviour, accelerated economic growth due to increased efficiency, or the inclusion of new technology sectors.

One major systemic phenomenon affecting ICT sustainability is the rebound effect. Originally described in the context of energy economics, the rebound effect occurs when efficiency

improvements lower the cost of a service, leading to increased consumption that partially or fully offsets the environmental gains (Hilty et al., 2006). A similar concept is the Jevons' Paradox mentioned by Eeckhout (2025), where the rebound effect is so strong that it fully cancels out or even reverses savings. This is visible in cloud computing because, as storage and processing become cheaper and more efficient, demand for data-intensive applications like streaming, gaming, or AI has increased. Rather than reducing overall energy consumption, innovations often stimulate greater use and increase environmental pressures. Understanding and addressing these systemic effects is imperative for truly sustainable software engineering. Models such as GREENSOFT model (Naumann et al., 2011) or the Karlskrona Manifesto (Becker et al., 2015) highlight the need to predict rebound dynamics during the design, deployment, and scaling of digital systems.

2.5 Need for a Holistic Sustainability Approach

Based on earlier sections, it is clear that hardware-only and software-only sustainability measures are inadequate and highlight the need for a comprehensive system-wide approach to sustainability in ICT. Addressing isolated parts of the problem will not suffice if broader lifecycle and system impacts are ignored (Becker et al., 2015; Lau et al., 2014). A truly sustainable digital ecosystem requires the consideration of both reduced energy consumption during operation and the embodied energy for hardware.

Models like the GREENSOFT model (Naumann et al., 2011) propose embedding sustainability principles throughout the entire SDLC, from requirements engineering through to disposal and recycling phases. Similarly, the Karlskrona Manifesto for Sustainability Design (Becker et al., 2015) calls for sustainability to be addressed as a foundational design concern alongside commonplace principles like security, usability, and performance. These perspectives argue that sustainability should be a primary quality attribute tied directly to how a system is conceived, built, and maintained.

3. Foundations of Green Software Engineering

3.1 Defining Green and Sustainable Software Engineering

Early ideas of sustainability in software stemmed from the reasoning that while software was nonphysical, it carried environmental impacts through its design, development, and lifecycle interactions with hardware systems. Dick et al. (2010) provided one of the earliest formal definitions of "Green and Sustainable Software", describing it as software that, post-development, deployment, and usage, has a positive effect on sustainable development regardless of the direct or indirect negative impacts it may generate.

In parallel, a separate but complementary concept, "Green and Sustainable Software Engineering", emerged to focus on the process rather than the product. While green software emphasises the characteristics and impacts of the final software product, green software engineering refers to the embedding of sustainability principles throughout the software development lifecycle (Naumann et al., 2011; Becker et al., 2015). This includes practices such as setting sustainability-oriented requirements, designing modular and resource-efficient architectures, optimising programming for energy consumption, and planning for long-term maintainability to reduce system obsolescence. Frameworks like the GREENSOFT Model (Naumann et al., 2011) and principles outlined in the Karlskrona Manifesto stress that sustainability should be regarded as a continuous engineering objective, comparable to primary attributes like security or usability. Understanding the distinction between sustainable software and sustainable software engineering is crucial for advancing green practices in ICT. Without addressing either property of the software product or the processes in its creation, you risk sustainability interventions being flawed or superficial.

3.2 Green by IT versus Green In IT

A focal distinction within the discussion on sustainable software is the difference between "Green by IT" and "Green in IT." This terminology, similar to what is mentioned in Section 3.1, introduced by Dick et al. (2010) and later expanded on by Naumann et al. (2011), clarifies the dual roles that software can play in environmental sustainability. "Green by IT" refers to the development of software systems that enable or support sustainability in other domains. Examples of this would be smart grid optimisation platforms, transportation management systems that minimise fuel use, or agricultural monitoring applications that enhance resource efficiency (Mahmoud & Ahmad, 2013; Albertao et al., 2010).

Conversely, "Green in IT" focuses on making the ICT sector itself more environmentally sustainable by improving the energy and resource efficiency of software systems internally (Dick et al., 2010). This encompasses practices like optimising algorithmic structures to reduce computation loads, designing software architectures with longevity and adaptability in mind, and minimising the indirect environmental impacts of ICT operations. Frameworks like the Karlskrona Manifesto (Becker et al., 2015) stress that "Green in IT" efforts are imperative as the ICT industry's growth trajectory threatens to negate sustainability gains made elsewhere if left unchecked. Understanding this distinction is crucial for properly framing sustainable software proposals.

3.3 Major Frameworks and Models

Several foundational models have been proposed to integrate sustainability considerations across the software development lifecycle. The GREENSOFT Model (Naumann et al., 2011) is among the most commonly used, offering a reference model that embeds sustainability assessments and actions into

every stage of development, deployment, usage, and disposal. Complementing this, Shenoy and Eeratta (2011) introduced the Green Software Development Model, as seen in Figure 2, emphasising practical steps to minimise energy consumption and material usage during development. Mahmoud and Ahmad (2013) further proposed the Green Model for Sustainable Software Engineering, which added a gold-orientated requirements engineering phase to ensure sustainability is addressed in earlier stages before running through the rest of the lifecycle. These models collectively highlight the importance of integrating environmental considerations systematically, not merely optimising final software products.

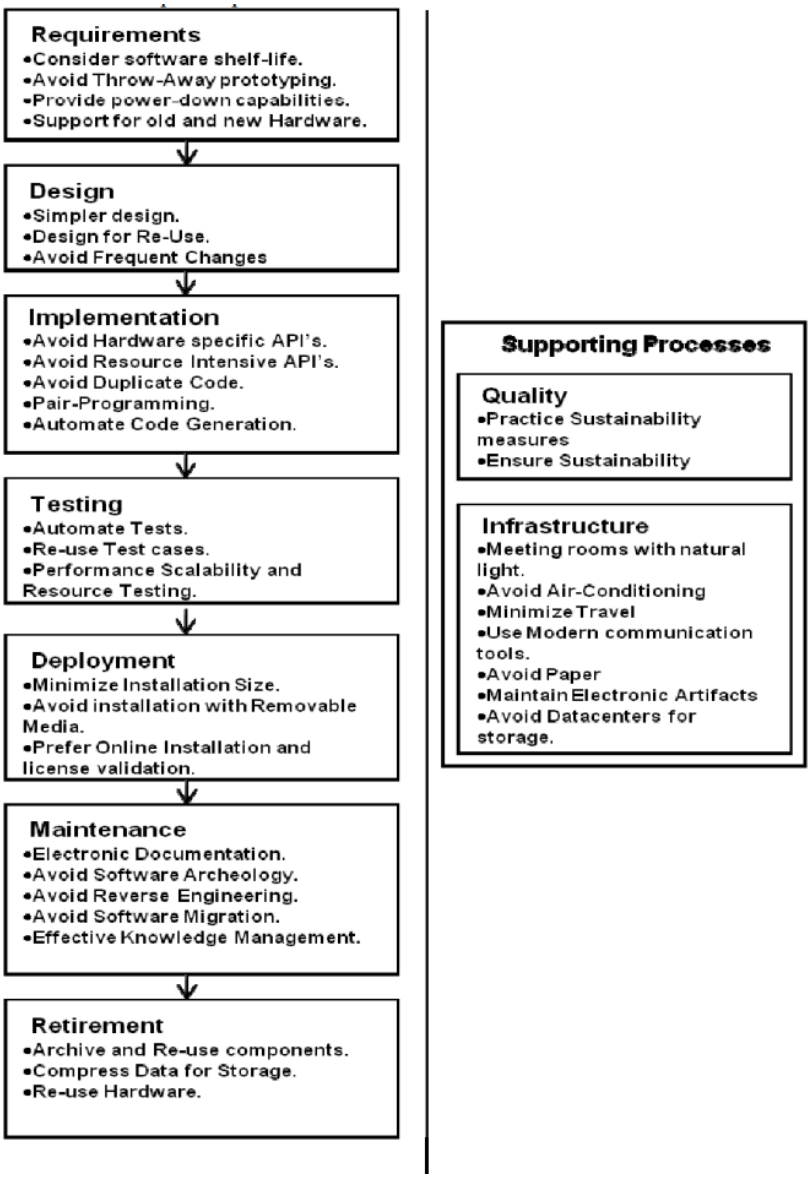


Figure 2 - The Green Software Development Life Cycle Model (Shenoy & Eeratta, 2011)

Beyond process-focused frameworks exist philosophical and evaluative models that expand the perspective of contribution sustainability measures to software. The Karlskrona Manifesto for Sustainability Design (Becker et al., 2015) establishes key principles that underpin much of modern systems thinking in sustainable software engineering. Venters et al. (2014) proposed a framework based on the story "The Blind Men and the Elephant" to assess software sustainability by a method of trial and error, as fragmented perspectives can lead to hindered holistic evaluation. Albertao et al. (2010) contributed a Sustainability Performance Measurement Model, introducing specific metrics to evaluate environmental impacts alongside traditional project management outcomes.

Recent efforts have further defined more diverse models of sustainability. Although earlier contributions like the Green IT Readiness Model by Molla et al. (2009) focused on an organisation's preparedness for eco-sustainability, more recent work has proposed new methods to embed sustainability into process phases. Valmohammadi and Mortaz (2023) proposed a Conceptual Green Process Model, using a mixed methods approach to systematically embed sustainability practices into process design. These models illustrate the breadth of approaches to sustainable software engineering, ranging from detailed lifecycle frameworks to organisational and cultural readiness assessments. However, a common challenge is still present as a lack of empirical testing in industrial contexts and standardised sustainability metrics across projects remain limited, highlighting the gaps that future research must address.

3.4 Challenges in Defining Metrics

Despite a growing interest in sustainable software engineering among researchers, establishing metrics and standards to evaluate sustainability remains a greater challenge. Early attempts such as the Sustainability Performance Measurement framework proposed by Albertao et al. (2010) suggested a form of integrating environmental considerations into traditional software project evaluations, but limited adoptions have been made. Models like GREENSOFT (Naumann et al., 2011) and the Green Model for Sustainable Software Engineering (Mahmoud & Ahmad, 2013) have included metrics like carbon footprint, energy consumption during execution, and hardware longevity, but a set of universally accepted indicators has not emerged yet. Venters et al. (2014) argued that coherent measurement efforts cannot be introduced without properly considering systemic impacts like energy efficiency.

Several factors contribute to this fragmentation. First, sustainability impacts span multiple dimensions: economic, environmental, and social, which makes adoption inherently more difficult to quantify compared to traditional software quality attributes like performance or security (Becker et al., 2015). Second, contextual factors such as deployment environments, usage patterns, and user behaviours can drastically alter the outcomes of sustainable software systems (Hilty et al., 2006; Kern, 2017). While tools like Greenalytics (Zapico et al., 2010) or the Green Tracker (Amsel & Tomlinson, 2010) offer valuable visualisation methods for raising awareness, there is a lack of standardisation across different software categories. Without a consistent and widely recognised metric, efforts to embed sustainability into software engineering will ultimately fall short.

4. Software Development Life Cycle

The software development life cycle (SDLC) represents a sequential structure for software systems, involving its creation, deployment, and eventual retirement. As software becomes more integrated with energy-intensive infrastructures and device ecosystems, the environmental sustainability of each phase in the SDLC has gained prominence. Industry specialists and researchers have responded to this issue with structured models like the GREENSOFT model (Naumann et al., 2011) as seen in Figure 3, the Green Software Development model (Shenoy & Eeratta, 2011), and the Green Model for Sustainable Software Engineering (Mahmoud & Ahmad., 2013) that embed sustainability features throughout the software life cycle. This section will explore the sustainability implications across the requirements, design, development, testing, deployment, maintenance, and disposal phases and discuss the metrics proposed to assess sustainable software practices.

4.1 Requirements and Design Stages

Embedding sustainability into software begins most effectively at the requirements and design stages. Several models showcase that early interventions offer the greatest leverage for reducing environmental impacts throughout a system's life cycle (Naumann et al., 2011; Mahmoud & Ahmad, 2013). Promoted by Mahmoud and Ahmad (2013), using Goal-Orientated Requirements Engineering (GORE) approaches will involve explicit incorporation of sustainability objectives into functional and non-functional requirements. This involves setting early goals such as minimising energy consumption, supporting backwards compatibility, and reducing update frequency. Similarly, Chitchyan et al. (2016) highlight that requirements engineering that is sustainability-aware must consider trade-offs between immediate system performance and long-term environmental impacts. Similarly, Duboc et al. (2019) state that raising early awareness in requirements engineering is a necessity, as negative factors often arise in early stages of software development, so it is key that sustainability impacts are identified. Without addressing these concerns early on, opportunities for later sustainability-themed optimisation are diminished greatly.

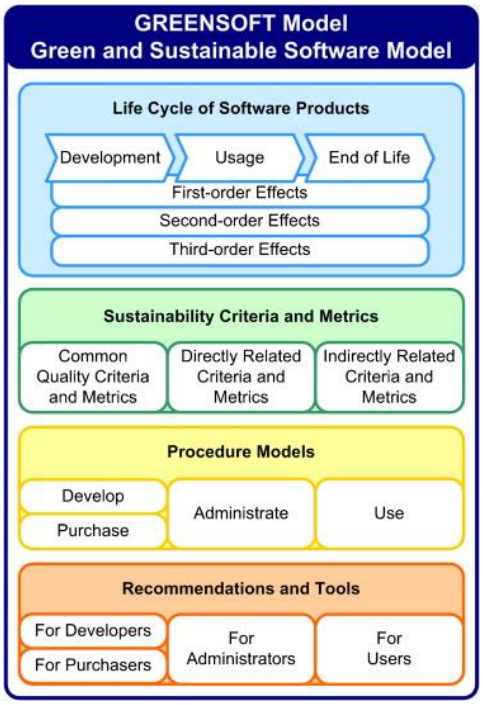


Figure 3 - The GREENSOFT Model, a reference model for "Green and Sustainable Software." (Naumann et al, 2011)

At the design stage, sustainable software engineering models encourage principles such as modularity, simplicity, and reusability. According to the GREENSOFT Model (Naumann et al., 2011) and the Green Software Development Model (Shenoy & Eeratta, 2011), by using modular architectures for maintenance and selective upgrading, the likelihood of whole system replacement and e-waste generation is reduced. In addition, careful design can optimise user interaction patterns and background processing demands, reducing energy usage (Venters et al., 2014). By embedding sustainability objectives into both requirements and architecture, software projects can proactively align environmental stewardship with core system qualities like usability, scalability, and reliability.

4.2 Development and Testing Stages

During the development phase, sustainability concerns shift from abstract goals to concrete programming practices. Energy-aware coding strategies, such as minimising computational complexity, optimising memory management, and reducing unnecessary database queries, are just a few that are critical to ensuring that applications operate efficiently (Shenoy & Eeratta, 2011; Mahmoud & Ahmad, 2013). The Green Software Development Model (Shenoy & Eeratta, 2011) particularly accentuates adopting coding standards that prioritise low-energy operations, favouring simple algorithms and leaner processing paths. Automated code analysis tools can assist developers in identifying hotspots of energy in code early on. Pair programming and agile code reviews, if supplemented with proper sustainability criteria, can further promote greener development practices by embedding awareness directly into team workflows (Becker et al., 2015).

Testing stages offer further opportunities to assess and improve sustainability before deployment. Tools such as Joulemeter (Kansal et al., 2010) enable developers to measure the energy consumption of individual software components during testing, providing actionable feedback on where optimisations are needed. Mahmoud and Ahmad (2013) argue that integrating sustainability metrics into automated testing, such as energy usage per test case or carbon impact per function, could reveal inefficiencies that traditional performance or validation testing may overlook. Moreover, using cloud-based automation testing environments can reduce unnecessary infrastructure emissions, with the assumption that careful resource provisioning is defined (Zapico et al., 2010). An example of an early tool designed to visualise the environmental impact of web applications is Greenalytics, shown in Figure 4. Although in beta form, it demonstrates the growing perception that software systems carry hidden but measurable carbon costs.

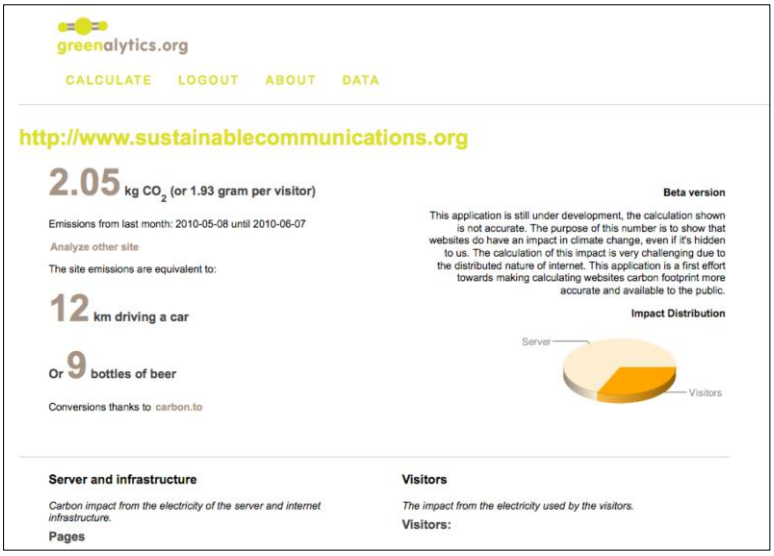


Figure 5 – Greenalytics.org beta from website estimating carbon emissions (adapted from Zapico et al, 2010)

4.3 Deployment, Maintenance and Disposal Stages

The deployment phase introduces sustainability challenges that are often overlooked but are vital for minimising software's environmental footprint. GREENSOFT (Naumann et al., 2011) and the Green Software Development Model (Shenoy & Eeratta, 2011) both highlight deployment optimisation strategies, such as reducing installation package sizes, using centralised/cloud-based deployments, and lessening redundant data transfers. Large installation files and frequent updates can strain network infrastructure unnecessarily and increase energy consumption, particularly when scaled across millions of devices. Packaging software efficiently, compressing resources, and offering selective modular downloads help to mitigate these impacts (Mahmoud & Ahmad, 2013). Furthermore, considering energy-efficient hosting solutions can significantly lower the emissions associated with software operation post-deployment (Zapico et al., 2010).

Sustainable maintenance and disposal practices aim to extend software longevity and reduce forced obsolescence of hardware. Mahmoud and Ahmad (2013) emphasise the use of sustainable maintenance strategies, including patching, that avoid bloated codebases and retain operational efficiency over time. Disposal considerations are often neglected in software projects as they involve planning for the decommissioning of software systems to minimise the generation of e-waste tied to hardware dependencies. Legacy hardware can be repurposed for less resource-intensive operations, or software can be optimised to extend the lifespan of older devices and align with circular economy practices at the same time.

4.4 Metrics for Sustainable Software

Quantifying metrics within the SDLC requires a multidimensional perspective. Naumann et al. (2011) proposed two metric types across the direct first-order impacts and the indirect second and third-order impacts. The metrics issued in the first order would suggest direct energy usage during execution and carbon footprint during the development and deployment phases of the SDLC, while second- and third-order effects address indirect and systemic effects. Second-order effects would involve how software helps reduce the need for physical processes, like virtual meetings replacing travel. Third-order metrics would involve changes in operational behaviour or observed device usage patterns because of software availability. Naumann et al. (2011) proposed two indicators of green performance: hardware compatibility longevity and the carbon footprint of the lifecycle of a software package.

Other researchers have proposed complementary metrics to capture different aspects of sustainability. Mahmoud and Ahmad (2013) proposed tracking installation package size and update frequency, arguing that smaller, less frequent updates reduce resource consumption and extend hardware viability. Shenoy and Eeratta (2011) pointed out energy consumption per function as a practical metric that is actionable at the development stage. In addition to developing metrics, efforts have been made to introduce sustainable labelling identifiers for software products, aiming to enhance user awareness and transparency about the environmental impact of digital services (Kern et al., 2015). Additional tools like Joulemeter (Kansal et al., 2010), which estimates the energy consumption of virtual machines and individual software processes, and the Green Power Indicator (Zapico et al., 2010), a browser plugin that visualises the energy impact of website usage, provide partial but useful measurements outside traditional SDLC evaluation processes. However, Venters et al. (2014) argue that fragmented and narrowed metrics risk missing systemic impacts unless altered by broader sustainable frameworks.

Overall, while diverse measurement techniques have emerged, the lack of standardised sustainability benchmarks across projects and industries remains a massive barrier. Future research must address the need for a comprehensive, context-sensitive metric that evaluates both operational energy use and lifecycle behaviour and social norms of software sustainability.

5. Software Abandonment and Large-Scale Failures

5.1 Causes of Abandonment

Large-scale software project failures often arise from a combination of technical, managerial, and strategic misalignments. Albertao et al. (2010) and Drummond (1999) both emphasise that runaway complexity, scope creep, inadequate requirements gathering, and changing stakeholder demands are common drivers of project escalation and eventual ruin. Poor risk management, weak stakeholder communication, and unrealistic timelines frequently complicate these issues and lead to the system failing to meet original goals or becoming obsolete before completion. From a sustainability viewpoint, such failures are a result of not representing environmental and economic costs. Abandoned systems consume resources during their development, generate emissions from their infrastructure use, and finally contribute heavily to e-waste from discarded hardware and software components (Venters et al., 2014; Becker et al., 2015).

Several historic cases illustrate these challenges vividly. The Customer Account Data Engine (CADE) project, which ran from 2000 to 2009 in the United States and intended to modernise tax processing, faced extensive delays and system integration issues, leading to eventual abandonment and major resource wastage. Similarly, the CSIO Portal in Canada ran from 1999 to 2006. They struggled with constant changing system requirements, security concerns, and poor architectural planning, leading to significant wasted budget. Both examples highlight sustainability failures that are not only technical but also systemic blunders, emerging when projects fail to understand long-term operational viability and adaptability. Recognising these patterns provides critical context for understanding the failures of other prominent projects such as the FBI's Virtual Case File (VCF), the UK e-Borders system, and the Taurus project, which will be discussed in the following sections.

5.2 FBI Virtual Case File

The FBI VCF project was launched in 2000 with the objective of modernising the agency's outdated paper-based case management system. The system was intended to allow agents and analysts to electronically store, retrieve, and share intelligence data more effectively and efficiently across the organisation (Marchewka, 2014). Over its development, the VCP project underwent many major revisions, expanding beyond its original scope and requiring frequent redesigns. By 2005, after approximately 170 million United States Dollars (USD) had been spent, the project was abandoned before reaching operational status. The abandoned VCF infrastructure was eventually replaced by a separate system, the Sentinel project.

5.3 UK e-Borders

The UK e-Borders program, initially formed in 2003, aimed to create an advanced passenger information system capable of screening individuals before their arrival into the United Kingdom. Managed initially by the UK Home Office and various private contractors, the project was envisioned with ambitions of integrating immigration, customs, and security data across multiple agencies to improve border security and operational efficiency (Alami, 2016a). The scope of e-Borders includes collecting and analysing extensive passenger information from airlines and other travel providers, enabling risk-based assessments in real time.

Throughout its development, the program underwent multiple internal structural changes, including contract shifts, policy changes, and integration attempts with legacy systems. Despite significant investments, reportedly exceeding 830 million British pounds (GBP), the e-Borders system was never

fully deployed as envisioned. By 2015, much of the project has been dismantled or absorbed into successor projects.

5.4 Taurus

The Transfer and Automated Registration of Uncertified Stock (Taurus) project, launched by the London Stock Exchange in the late 1980s, was aimed at automating the settlement and registration of share transactions, replacing the paper-based system that was existing at the time. It was designed to create an electronic platform that would streamline trading operations, reduce administrative costs, and enhance market efficiency (Drummond, 1999).

Initially budgeted at six million GBP, Taurus quickly expanded in scope and complexity, as demands from various financial institutions requested additional features and customisations. The project experienced numerous delays, continual revisitations, and technical integration difficulties. After several years of development and an estimated 400 million GBP in accumulated costs across stakeholders, the Taurus project was formally abandoned in 1993 without delivering a working system.

5.5 Reason for project failure

Across the FBI VCF, UK e-Borders, and Taurus projects, several reoccurring causes of failure are evident. One major issue was uncontrolled scope expansion, where initial project goals were continually revisited without corresponding amendments in planning or design (Marchewka, 2014; Alami, 2016a; Drummond, 1999). In all three cases, shifting requirements overwhelmed the original system designs, leading to great complexity and technical instability. Poor stakeholder management further compounded these problems, with communication gaps between developers, end users, and sponsoring agencies meaning that systems evolved without fully addressing operational needs. Technical overstretching also played a significant role as projects attempted to integrate outdated legacy systems or deploy unproven techniques without sufficient prototyping. Observations that can be noticed from these three projects align with broader patterns identified in ICT project failure literature, which highlight insufficient risk management, unrealistic timelines, and lack of adaptability to key contributors (Alami, 2016b).

A sustainability-oriented approach could have significantly mitigated these failures. Early modular design, with emphasis on flexibility and incremental deliverables, would have allowed the projects to adapt more gracefully to the changing requirements without requiring complete system overhauls. GORE could have helped clarify essential versus non-essential features, keeping the projects aligned with core needs (Mahmoud & Ahmad, 2013). Sustainability metrics, such as monitoring infrastructure energy efficiency, might have triggered earlier interventions to reduce unnecessary resource consumption (Albertao et al., 2010; Zapico et al., 2010). Embedding sustainability principles such as adaptability, minimal viable architecture, and lifecycle impact awareness into the software engineering process would have offered a path forward that would have prevented the failure of the projects. These failures carried a substantial environmental cost, where resources used in development, such as electricity, hardware infrastructure, and computation, were expended and ultimately wasted. Understanding these fatal flaws of hidden environmental impacts further underscores the need to implement sustainability practices into the earliest stages of software system design.

6. Green Software Adoption in Industry Challenges

Despite growing awareness of the environmental footprint of software systems, integration of green software practices into mainstream industry remains limited. While academic models such as GREENSOFT (Neumann et al. 2011) and the Green Software Development Model (Shenoy & Eeratta 2011) offer detailed pathways for embedding sustainability into the software lifecycle, practical methods of adoption for industry are obstructed by a broad set of real-world constraints. These include economic pressures, lack of awareness, insufficient tooling, and increased misconceptions about trade-offs between sustainability, performance, and profitability. This section will discuss the key barriers inhibiting adoption of sustainable software engineering practices in industry.

6.1 Economic and Organisational Constraints

One of the most significant impediments to the adoption of green software engineering is the lack of financial incentives. Organisational profits are often tied to short-term financial performance rather than long-term sustainability benefits, particularly when the cost savings from energy-efficient software are not immediately visible (Mahmoud & Ahmad, 2013). Benefits, such as reduced energy consumption, extended hardware lifespan, and future compliance with environmental regulations, are difficult to quantify within conventional return on investment models. Furthermore, sustainability is often treated as a secondary concern rather than a core project initiative, leading to minimal resource allocation during planning stages (Valmohammadi & Mortaz, 2023). Software development structures that prioritise speed through agile sprints or continuous integration/development further discourage sustainability improvements. Additional factors, such as the need to rethink infrastructure, retrain personnel, or potentially delay release cycles, also deter organisations from integrating sustainable practices.

6.2 Lack of Awareness and Tooling

Another major challenge lies in the low level of awareness among software professionals regarding the environmental impacts of software decisions. Many developers, project managers, and even company executives fail to recognise and acknowledge sustainability as a relevant design concern (Kern, 2017). Broader survey results further illustrate the gap between awareness and practical knowledge in the ICT sector. As shown in Table 1, while knowledge of green computing exists among various groups, consistent sustainable practices remain lacking (Kern, 2017). As a result, few organisations will include green metrics in performance evaluations or in software quality benchmarks. As a result, sustainability metrics are rarely implemented into software quality benchmarks or performance evaluations. Tools like Joulemeter (Kansal et al., 2010), Greenalytics (Zapico et al., 2010), and the Green Power Indicator (Zapico et al., 2010) have been developed to measure energy usage, but they are not widely integrated into mainstream development environments. Mahmoud and Ahmad (2013) argue that green software tooling is typically fragmented and often seen as a supplementary aspect rather than essential. This lack of sustainability indicators and benchmarks increases the difficulty of the problem. Although green performance indicators have been proposed in academic literature (Neumann et al., 2011), they are not widely enforced by industry professionals, which leaves organisations without proper references for measurements.

Target group	Objective and focus	Findings	Source
Programmers	Do programmers know about energy consumption of *software* and how to reduce it although such issues are not addressed in education or trainings?	Programmers had limited knowledge of energy efficiency, lacked knowledge of the best practices to reduce energy consumption and were unsure about how software consumes energy	Pang et al. (2016)
Practitioners	Relationship between research community and practitioners: same interests? Asking for the practitioners' perspective to get a motivation and guide for green *software* engineering research	Practitioners care about energy but are not as effective as they could be at creating efficient applications	Manotas et al. (2016)
Students in Mauritius	Analyze the awareness of green computing (focus: *hardware*) among students	Students have a moderate knowledge about green computing but their everyday green computing practices are not satisfactory	Dookhitram et al. (2012)
Students in Malaysia	*Hardware* focus: usage and selling of computers and related resources	Students have an average level awareness about green computing and their everyday practices are not satisfactory	Selyamani and Ahmad (2015)
Buyer and supplier	Analyze people's awareness of *software* engineering issues	People are aware of the importance of energy efficient software, but lack information and resources to engage this topic	Ferreira (2011)
ICT Managers	How do ICT managers implement energy efficient methods (focusing on *hardware* issues) in their organizations?	Just over a third of organisations in Europe do not implement green IT practices	Kogelman (2011)

Table 1—Overview of studies addressing the awareness of green computing issues (adapted from Kern, 2017)

6.3 Cultural and Technical Misconceptions

A persistent cultural barrier is the perception that green software engineering compromises performance, innovation, or usability. For instance, optimising code for energy efficiency is often assumed to reduce computational speed or limit design richness (Becker et al., 2015). There is also significant resistance to change in many engineering environments, especially in agile or lean teams where flexibility and rapid iteration are highly valued. Introducing sustainability as a design concern is sometimes viewed as an impediment to these workflows. Moreover, even in organisations that are aware of environmental issues, sustainability tends to be ranked lower than traditional project priorities such as reliability, scalability, or maintainability (Naumann et al., 2011). These misconceptions perpetuate a cycle where sustainability is seen as optional rather than a fundamental quality attribute.

6.4 Trade-offs and Complexity in Real-World Projects

Even when sustainability is acknowledged as being vital, real-world initiatives still require challenging trade-offs for practical implementation. Project managers and developers frequently find it difficult to strike a balance between conflicting goals: sustainability goals must coexist with client deadlines, financial limitations, user experience requirements, and system performance standards (Venters et al., 2014). Often, especially in fiercely competitive business settings, short-term functional needs take precedence over long-term sustainability objectives. According to Kern (2017), this tension is made worse by the lack of sustainability evaluation tools, which prevent teams from measuring and defending green design choices objectively inside routine processes. In addition to better tools, project management techniques must change to incorporate sustainability standards from the very beginning of planning to overcome these real-world obstacles and make sure they are not compromised by operational demands.

7. Summary

This literature review examined how sustainability and software engineering are changing together, emphasising how crucial it is to incorporate environmental factors into every stage of the software development life cycle. At first, ICT sustainability initiatives were mostly concerned with hardware efficiency, but as the environmental impact of software systems increased, focus was switched to sustainable and green software engineering techniques. From requirements collection to deployment and disposal, many methods of integrating sustainability are illustrated by foundational models, including GREENSOFT, the Green Software Development Model, and the Green Model for Sustainable Software Engineering, to name a few. The FBI Virtual Case File, the UK e-Borders system, and the Taurus project are just a few instances of large-scale software failures that illustrate the significant resource and environmental consequences linked to inadequate early lifecycle planning and technical misalignment. Furthermore, economic pressures, ignorance, fragmented tooling, and cultural misunderstandings regarding trade-offs between sustainability and essential software features continue to hinder industry adoption of sustainable practices.

There are still large gaps in both research and practice, despite an increase in scholarly contributions. Software sustainability metrics are still dispersed and lack industry-wide standardisation, which makes it difficult to compare and evaluate projects in a meaningful way. Furthermore, current frameworks frequently fail to adequately handle systemic impacts like the rebound effect or the more extreme Jevons' paradox, which occurs when enhanced efficiency results in higher overall resource use. Adoption is made more difficult by practical obstacles in real-world initiatives, such as strict commercial deadlines and a lack of integration of sustainability indicators into routine procedures. Green software engineering runs the risk of staying a novel concept rather than becoming the standard in the industry if sustainability is not treated as a fundamental quality factor rather than an optional condition.

The goal of future work should be to create usable, standardised sustainability criteria that are easily included in regular software engineering procedures. Building solid, scalable solutions will need a transdisciplinary approach, where environmental, social, economic, and software engineering can coexist. Priority should be given to educational programmes aimed at increasing awareness among executives, project managers, and developers. Putting sustainability at the core of software engineering is both an opportunity and a responsibility to make sure that technical advancements promote rather than impede global environmental goals as digitalisation continues to transform society.

References

Alami, A. (2016a). The UK e-Borders Project Failure. In *PM World Journal The UK e-Borders Project Failure*. https://pmworldlibrary.net/wp-content/uploads/2016/03/pmwj44-Mar2016-Alami-uk-eborders-project-failure-featured-paper.pdf

Alami, A. (2016b). Why Do Information Technology Projects Fail? *Procedia Computer Science*, *100*, 62–71. https://doi.org/10.1016/j.procs.2016.09.124

Albertao, F., Xiao, J., Tian, C., Lu, Y., Zhang, K. Q., & Liu, C. (2010). Measuring the Sustainability Performance of Software Projects. *2010 IEEE 7th International Conference on E-Business Engineering*, 369–373. https://doi.org/10.1109/icebe.2010.26

Amsel, N., & Tomlinson, B. (2010). Green tracker. *10 Extended Abstracts on Human Factors in Computing Systems*. https://doi.org/10.1145/1753846.1753981

Becker, C., Chitchyan, R., Duboc, L., Easterbrook, S., Penzenstadler, B., Seyff, N., & Venters, C. C. (2015). Sustainability Design and Software: The Karlskrona Manifesto. *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*. https://doi.org/10.1109/icse.2015.179

Bekaroo, G., Bokhoree, C., & Pattinson, C. (2013). Towards Green IT Organisations: A Framework for Energy Consumption and Reduction. *ResearchGate*, *8*(3). https://doi.org/10.18848/1832-3669/CGP/v08i03/56290

Ceurstemont, S. (2024). Controlling AI's Growing Energy Needs. *Communications of the ACM*, *68*(3), 15–17. https://doi.org/10.1145/3703788

Chitchyan, R., Becker, C., Betz, S., Duboc, L., Penzenstadler, B., Seyff, N., & Venters, C. C. (2016). Sustainability design in requirements engineering. *Proceedings of the 38th International Conference on Software Engineering Companion - ICSE '16*. https://doi.org/10.1145/2889160.2889217

Deng, Q., & Ji, S. (2015). Organizational Green IT Adoption: Concept and Evidence. *Sustainability*, *7*(12), 16737–16755. https://doi.org/10.3390/su71215843

Dick, M., & Naumann, S. (2010). *Enhancing Software Engineering Processes towards Sustainable Software Product Design* (pp. 706–715). EnviroInfo. https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=98d04de0318b252f273e3cf36cc385253542b924

Dick, M., Naumann, S., & Kuhn, N. (2010). *A Model and Selected Instances of Green and Sustainable Software*. 248–259. https://doi.org/10.1007/978-3-642-15479-9_24

Drummond, H. (1999). Are we any closer to the end? Escalation and the case of Taurus. *International Journal of Project Management*, *17*(1), 11–16. https://doi.org/10.1016/s0263-7863(97)00074-4

Duboc, L., Betz, S., Penzenstadler, B., Kocak, S. A., Chitchyan, R., Leifler, O., Porras, J., Seyff, N., & Venters, C. C. (2019, September 1). *Do we Really Know What we are Building? Raising Awareness of Potential Sustainability Effects of Software Systems in Requirements Engineering*. IEEE Xplore. https://doi.org/10.1109/RE.2019.00013

Eeckhout, L. (2025). The Sustainability Gap for Computing: Quo Vadis? *Communications of the ACM*, *68*(3), 70–79. https://doi.org/10.1145/3699595

Hilty, L. M., Köhler, A., von Schéele, F., Zah, R., & Ruddy, T. F. (2006). Rebound effects of progress in information technology. *Poiesis & Praxis*, *4*(1), 19–38. https://doi.org/10.1007/s10202-005-0011-2

Kansal, A., Zhao, F., Liu, J., Kothari, N., & Bhattacharya, A. (2010). Virtual machine power metering and provisioning. *Symposium on Cloud Computing*. https://doi.org/10.1145/1807128.1807136

Kern, E. (2017). Green Computing, Green Software, and Its Characteristics: Awareness, Rating, Challenges. *Progress in IS*, 263–273. https://doi.org/10.1007/978-3-319-65687-8_23

Kern, E., Dick, M., Naumann, S., & Filler, A. (2015). Labelling sustainable software products and websites: Ideas, Approaches, and Challenges. *Advances in Computer Science Research*. https://doi.org/10.2991/ict4s-env-15.2015.10

Lau, L. M. S., Griffiths, M., Holmes, V., Ward, R., Jay, C., Dibsdale, C., Xu, J., Austin, J., & Venters, C. C. (2014). Software Sustainability: The Modern Tower of Babel. *RE4SuSy: Third International Workshop on Requirements Engineering for Sustainable Systems*, *1216*. https://www.researchgate.net/publication/303617265_Software_Sustainability_The_Modern_Tower_of_Babel

Mahmoud, S. S., & Ahmad, I. (2013). A green model for sustainable software engineering. *International Journal of Software Engineering and Its Applications*, *7*(4), 55–74. https://www.researchgate.net/publication/285762532_A_green_model_for_sustainable_software_engineering

Marchewka, J. T. (2014). The FBI Virtual Case File: A Case Study. *Communications of the IIMA*, *10*(2). https://doi.org/10.58729/1941-6687.1132

Mayr, H. C., Pinzger, M., & Kern, E. (2016). *The Development of an Eco-Label for Software Products - a Transdisciplinary Process?* (p. 1285). Gesellschaft für Informatik. https://cs.emis.de/LNI/Proceedings/Proceedings259/1285.pdf

Mocigemba, D. (2006). Sustainable Computing. *Poiesis & Praxis*, *4*(3), 163–184. https://doi.org/10.1007/s10202-005-0018-8

Molla, A., Cooper, V. A., & Pittayachawan, S. (2009). IT and eco-sustainability: Developing and validating a green IT readiness model. *ResearchGate*. https://doi.org/10.13140/2.1.5004.1124

Naumann, S., Dick, M., Kern, E., & Johann, T. (2011). The GREENSOFT Model: A reference model for green and sustainable software and its engineering. *Sustainable Computing: Informatics and Systems*, *1*(4), 294–304. https://doi.org/10.1016/j.suscom.2011.06.004

Penzenstadler, B. (2013). *Towards a definition of sustainability* in *and* for *software engineering*. https://doi.org/10.1145/2480362.2480585

Shenoy, Sanath. S., & Eeratta, R. (2011). Green software development model: An approach towards sustainable software development. *2011 Annual IEEE India Conference*. https://doi.org/10.1109/indcon.2011.6139638

Valmohammadi, C., & Mortaz Hejri, F. (2023). Designing a conceptual green process model in software development: A mixed method approach. *International Journal of Information Management Data Insights*, *3*(2), 100204. https://doi.org/10.1016/j.jjimei.2023.100204

Vartziotis, T., Dellatolas, I., Dasoulas, G., Schmidt, M., Schneider, F., Hoffmann, T., Kotsopoulos, S., & Keckeisen, M. (2024). Learn to Code Sustainably: An Empirical Study on Green Code

Generation. *Proceedings of the 1st International Workshop on Large Language Models for Code*, 30–37. https://doi.org/10.1145/3643795.3648394

Venters, C. C., Lau, L., Griffiths, M. K., Holmes, V., Ward, R. R., Jay, C., Dibsdale, C. E., & Xu, J. (2014). The Blind Men and the Elephant: Towards an Empirical Evaluation Framework for Software Sustainability. *Journal of Open Research Software*, *2*(1). https://doi.org/10.5334/jors.ao

Zapico, J. L., Turpeinen, M., & Brandt, N. (2010). *Greenalytics: a tool for mash-up life cycle assessment of websites.* ResearchGate. https://www.researchgate.net/publication/229037499_Greenalytics_a_tool_for_mash-up_life_cycle_assessment_of_websites

Appendix B

Official Ethics Approval Letter

THE UNIVERSITY OF WAIKATO
Te Whare Wānanga o Waikato

Te Wānanga Pūtaiao – Division of STEM
School of Engineering
Dr Megan Boston
Tel: +64 7 837 9459
Email: megan.boston@waikato.ac.nz
stem-ethics@waikato.ac.nz

+64 7 838 4144
waikato.ac.nz

Gate 1, Knighton Road
Hamilton 3240

Private Bag 3105
Hamilton 3240
New Zealand

29 July 2025

Aryan Thanki
At314@students.waikato.ac.nz
School of Computing and Mathematical Sciences

cc: Judy Bowen

Dear Aryan

Re: STEM_HREC(2025)#37

I refer to your resubmitted application submitted to the STEM Human Research Ethics Committee entitled "Sustainability as a Software Engineering Problem".

Your revised application has now been carefully considered by members of the STEM Human Research Ethics Committee, and I am pleased to offer formal approval for your research activities.

We encourage you to contact a member of the committee should issues arise during your data collection, or should you wish to add further research activities or make changes to your project as it unfolds.

We wish you all the best with your research.

Regards

Megan Boston

Dr Megan Boston, Convenor
Division of STEM - Human Research Ethics Committee